

**АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ПРОФЕССИОНАЛЬНАЯ
ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ МОСКОВСКИЙ МЕЖДУНАРОДНЫЙ
КОЛЛЕДЖ ЦИФРОВЫХ ТЕХНОЛОГИЙ «АКАДЕМИЯ ТОП»**

**ОТЧЕТ ПО
УЧЕБНОЙ ПРАКТИКЕ**

Обучающийся Минко Галина Станиславовна

Фамилия, Имя, Отчество

Образовательная программа (код и наименование): 09.02.07 Информационные системы и программирование

Профессиональный модуль ПМ.02. ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ

№ группы 9/3-РПО-22/1-III

Место прохождения практики: АНО ПОО Московский международный колледж цифровых технологий “Академия ТОП”

Срок прохождения практики: с 26.02.2025 по 30.04.2025

1. Описание Профильной организации

Полное и сокращенное наименование организации: АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ДОПОЛНИТЕЛЬНОГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «АКАДЕМИЯ ТОП» (АНО ДПО «АКАДЕМИЯ ТОП»)

Учредитель: *Общество с ограниченной ответственностью «Компьютерная Академия ТОП»*

Место нахождения: *121170, город Москва, проспект Кутузовский, дом 36, строение 2, эт/пом/ком 4/1/20*

2. Содержание и результаты практики

В рамках практики я занималась реализацией проекта — многопользовательской 2D-игры NEURO SOIL, разработанной на Unity с использованием сетевой библиотеки Mirror. Основной задачей было не просто создать игру, а интегрировать в неё сетевой модуль, организовать взаимодействие игроков и синхронизацию объектов в сетевой среде.

Проект оказался заметно сложнее, чем предполагалось, и сопровождался высокой нагрузкой — в том числе из-за интенсивной самостоятельной проработки, смены технологий и работы с неизвестными инструментами.

Однако, несмотря на сложности, я получила значимый опыт:

- Освоила подходы к самостоятельному решению технических задач,
- Разобралась в архитектуре сетевых игр,
- Научилась настраивать многопользовательский режим, корректную синхронизацию игроков и объектов.

Эта практика стала возможностью выйти за рамки базовых знаний и применить навыки в контексте реального проекта.

3. Общие впечатления о практике. Степень соответствия знаний, умений и практических навыков, оцениваемых в рамках практики профессиональным компетенциям, основным видам деятельности, предусмотренным ФГОС СПО и уровням квалификаций в соответствии с профессиональными стандартами.

Практика стала настоящим испытанием, но одновременно — и шагом вперёд в моём профессиональном развитии.

Полученные навыки, особенно в части работы с реальным техническим проектом и модульной интеграцией, соответствуют требованиям ФГОС СПО и профессиональным компетенциям.

Уровень практики был высоким, и именно благодаря этому я почувствовала переход от позиции студента к уровню начинающего разработчика.

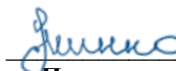
4. Ваши пожелания по организации и проведению практики.

Хотелось бы, чтобы в дальнейшем часть практики имела более чёткую структуру и сопровождалась поддержкой куратора по конкретным техническим вопросам.

Было бы полезно также подключать специалистов, знакомых с предметной областью, выбранной студентом, и формировать небольшие проектные группы.

Это позволило бы более эффективно решать прикладные задачи, развивать командную работу и получать более качественную обратную связь.

Обучающийся


Подпись

/Г. С. Минко/
И.О. Фамилия

Содержание

ВЕДЕНИЕ.....	5
1. Анализ предметной области	6
1.1 Анализ существующих аналогов	6
1.2 Спецификация требований для клиентского приложения	8
1.2.1 Функциональные требования	9
1.2.2 Нефункциональные требования	16
1.3 Диаграмма потока экранов	16
1.4 Выводы по главе	18
Глава 2. Проектирование	19
Архитектура взаимодействия	19
Компоненты клиентского приложения.....	20
Методология проектирования	21
2.1 Диаграммы пригодности.....	21
2.1.1 Прецедент «Играть».....	21
2.1.2 Прецедент «Авторизоваться».....	23
2.1.3 Прецедент «Изменить настройки»	24
2.2 Диаграммы последовательности	26
2.2.3 Прецедент «Изменить настройки»	28
2.3 Диаграммы классов	29
Основные компоненты:.....	29
2.3 Диаграммы классов	
Основные компоненты:.....	
Диаграмма классов: появление объектов на сцене.....	30
Диаграмма классов: процесс авторизации	31
2.4 Диаграмма базы данных	32
2.5 Выводы по главе	33
Глава 3. Реализация и тестирование	34

					09.02.07 7898.2025. У П.02.01						
Изм	Лист	№ документа	Подпись	Дата	О т ч е т п о у ч е б н о й п р а к т и к е ПМ.02			Лит	Лист	Листов	
Разработал	Минко Г. С.		30.04.25							3	62
Руководитель	Григорьев М.Ю.		30.04.25	АНО ДПО «АКАДЕМИЯ ТОП» 9/3-РПО-22/1-Ш							
Н.контроль	Григорьев М.Ю.		30.04.25								

Техническое задание для проекта HEURO COIL.....	34
1. Общие сведения	34
2. Цель проекта	34
3. Требования к проекту.....	35
3.1 Функциональные требования.....	35
3.2 Нефункциональные требования.....	35
4. Структура экранов и навигация	36
5. Особенности реализации	36
6. Разработка и анализ требований к программной среде.....	37
6.1 Средства разработки.....	37
6.2 Целевая платформа.....	37
6.3 Требования к системам разработки и запуска.....	38
6.4 Ограничения и особенности программной среды.....	38
3.1 Выбор инструментов.....	39
Выбор сетевой библиотеки: от NGO к Migor	39
4. Входные и выходные данные	40
4.1 Входные данные.....	40
4. Входные и выходные данные	41
4.1 Входные данные.....	41
4.2 Выходные данные	42
5. Реализация	42
5.1 Меню и интерфейс пользователя	42
5.2 Управление змейкой	43
5.3 Генерация еды.....	44
5.4 Хвост змейки	45
5.5 Сетевой менеджер и запуск.....	45
5.6 Завершение игры	46
5.7 Дополнительные компоненты	46
6. Тестирование	47
Заключение	48

					09.02.07 7898.2025.У П.02.01				
Изм	Лист	№ документа	Подпись	Дата	О т ч е т п о у ч е б н о й п р а к т и к е ПМ.02	Лит	Лист	Листов	
Разработал	Минко Г. С.			30.04.25			4	62	
Руководитель	Григорьев М.Ю.			30.04.25		АНО ДПО «АКАДЕМИЯ ТОП» 9/3-РПО-22/1-Ш			
Н.контроль	Григорьев М.Ю.			30.04.25					

ВЕДЕНИЕ

Игровая индустрия продолжает активно развиваться, и одной из самых устойчивых категорий остаются аркадные игры, в частности — вариации классической «Змейки». По данным аналитического ресурса [pr-su.ru](#) [1], ежедневно сайт [Snake.io](#) [2], предоставляющий доступ к одноимённой онлайн-игре, привлекает более 1,3 миллиона пользователей. Основной доход разработчиков таких проектов формируется за счёт монетизации через встроенную рекламу и микротранзакции. Это подтверждает устойчивый интерес аудитории к простым, но соревновательным форматам.

Целью настоящей учебной практики является разработка клиентской части 2D-сетевой игры «Змейка» с возможностью последующего масштабирования и интеграции дополнительных модулей — таких как таблица лидеров или внутриигровой рейтинг. В качестве движка используется **Unity**, а реализация сетевого взаимодействия выполнена с помощью библиотеки **Mirror**, обеспечивающей архитектуру «клиент–сервер» и поддержку различных режимов подключения (Host, Server, Client).

Структура работы напрямую соответствует решаемым задачам и этапам разработки.

В первой главе проводится обзор аналогичных проектов и анализ предметной области. Выделяются жанровые и технические особенности, определяются возможности масштабирования и доработки. На основе анализа формулируются требования к клиентской части и логике взаимодействия игроков.

Вторая глава посвящена проектированию структуры приложения, включая архитектуру компонентов, систему подключения, организацию сетевой синхронизации и взаимодействия объектов в сцене. Для описания логики используются структурные диаграммы и схемы компонентов.

В третьей главе кратко описывается реализация проекта в среде Unity, настройка компонентов сетевого взаимодействия, построение интерфейса и логики управления. Отдельное внимание уделяется пользовательскому меню, системе настройки аудио, подключению к серверу, а также реализации основных игровых событий. Завершается работа сборкой проекта, тестированием функционала и подготовкой сопровождающей документации.

В ходе выполнения учебной практики были достигнуты следующие результаты:

- разработано техническое задание на создание сетевой 2D-игры;
- выполнено проектирование архитектуры клиентской части и системы взаимодействия;
- реализован базовый игровой прототип с полноценной поддержкой сетевого режима;
- разработано пользовательское меню, система управления аудио и логика выбора режима подключения;
- проведено функциональное тестирование и оформлена итоговая документация.

					09.02.07 7898.2025. У П.02.01	Лист
						5
Изм	Лист	№ документа	Подпись	Дата		

1. Анализ предметной области

1.1 Анализ существующих аналогов

Для обеспечения актуальности и конкурентоспособности разрабатываемого программного продукта был проведён анализ популярных онлайн-игр, реализующих механику классической змейки в многопользовательском формате. Целью анализа стало выявление сильных и слабых сторон аналогов, а также поиск функциональных ниш и улучшений, которые могли бы быть реализованы в рамках собственной разработки.

Особое внимание было уделено играм **Snake.io** [2], **Wormax.io** [3] и **Slither.io** [5] — проектам, получившим широкую известность за счёт сочетания простой игровой механики, онлайн-взаимодействия и встроенной монетизации.

Результаты сравнительного анализа представлены в таблице 1.

Таблица 1 — Сравнение аналогов

Критерий сравнения	Snake.io [2]	Wormax.io [3]	Slither.io [5]
Количество просмотров страниц, млн человек/день	1.3	2.2	19.6
Монетизация	Реклама	Реклама	Реклама
Создание учётной записи	+	+	-
Общение с другими игроками	-	-	-
Поделиться результатом в соцсетях	+	+	+
Однопользовательский режим	-	-	+
Платные дополнения	+	+	-
Общая статистика игроков	+	+	-
Статистика в текущей игровой сессии	+	+	+
Ежедневные задания	-	+	-
Повышение ранга	-	+	-
Мини-карта	-	+	+
Выбор управления	+	-	-
Особые предметы	-	+	-
Создание своей модели персонажа	-	-	+
Указатель на лучшего игрока текущей сессии	+	-	-

					09.02.07 7898.2025. У П.02.01	Лист 6
Изм	Лист	№ документа	Подпись	Дата		

Выводы по результатам анализа

Анализ показывает, что даже у популярных реализаций игры “Змейка” остаются **недостатки**, которые могут быть устранены при разработке нового продукта. В частности:

- **Отсутствие чата или системы взаимодействия игроков** в большинстве аналогов снижает социальную составляющую.
- Возможности **персонализации персонажа** реализованы не во всех играх.
- **Единая таблица лидеров, статистика и система рангов** — значимый мотивирующий элемент, но он доступен не во всех версиях.

С учётом выявленных особенностей, в рамках разработки 2D-игры «Snake Online 2D» планируется реализовать следующие функции:

- Интеграция **таблицы лидеров**, отражающей лучшие результаты за сессию и за всё время;
- Возможность **выбора и настройки модели персонажа**;
- Расширение **сетевых возможностей** и проработка взаимодействия игроков (в перспективе);
- Поддержка **основных мобильных и десктопных платформ** через Unity;
- Внедрение **модуля аналитики** для сбора и отображения игровой статистики (по желанию).

На основании проведённого анализа была сформирована начальная **спецификация требований** к проекту, представленная в следующем разделе.

					09.02.07 7898.2025.У П.02.01	Лист
						7
Изм	Лист	№ документа	Подпись	Дата		

1.2 Спецификация требований для клиентского приложения

Пользователь клиентского приложения — это игрок, участвующий в многопользовательской сетевой 2D-игре, в которой он управляет персонажем (змеей) в ограниченном игровом пространстве. Основной задачей игрока является **сбор объектов (еды, бонусов)**, способствующих **увеличению размеров** персонажа и набиранию как можно большего количества очков.

Игра происходит в **режиме реального времени**, в одной игровой сессии одновременно участвует множество пользователей. Игроки могут **сталкиваться и взаимодействовать** друг с другом — например, блокировать движение, устраивать ловушки, пытаться вытеснить соперников с поля. Подобная механика требует высокой скорости отклика и стабильного сетевого соединения.

В процессе игры пользователь имеет доступ к следующим возможностям:

- Отслеживание **текущего результата (очки)**;
- Просмотр **таблицы лидеров** текущей игровой сессии;
- Ориентирование на **лучших игроков**, чтобы оценивать собственный прогресс;
- Использование **особых способностей** (например, временное ускорение).

После завершения игровой сессии (например, при столкновении с другим игроком или границей поля), игроку отображается **результат**, набранные очки и позиция в рейтинге. Далее пользователь может:

- Перезапустить игру;
- Ознакомиться с **общей таблицей лидеров**;
- Настроить внешний вид своего персонажа (если реализовано);
- В перспективе — поделиться результатом в социальных сетях или пригласить друзей в сессию.

Основные функциональные требования:

- Поддержка многопользовательской игры с **сетевым подключением**;
- Интерактивное отображение игровых событий и **2D-сцены**;
- Реализация логики **управления персонажем** в реальном времени;
- Обновляемая **таблица лидеров** (на основе локальной сессии и/или серверной статистики);
- Возможность **перезапуска** и перехода к другим режимам (по мере разработки).

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		8

Нефункциональные требования:

- Высокая **производительность и отклик** при сетевом взаимодействии;
- Совместимость с ОС **Microsoft Windows**;
- Использование **Unity** как основного игрового движка;
- Возможность **масштабирования** и внедрения дополнительных модулей (аналитика, авторизация, внутриигровой чат и т.д.).

Вся логика, связанная с расчётом очков, коллизиями, отображением интерфейса и отправкой сетевых запросов, реализуется на стороне клиента, с возможностью обращения к серверу для синхронизации данных и получения таблицы рекордов.

1.2.1 Функциональные требования

На рисунке 1.1 представлена **диаграмма прецедентов** разрабатываемого клиентского приложения.



Рисунок 1.1 — Диаграмма прецедентов

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		9

Прецедент 1. Зарегистрироваться

Цель:

Пользователь создаёт новую учётную запись в системе для получения доступа к основному функционалу игры.

Предусловия:

Пользователь находится в **основном меню** приложения.

Основной сценарий:

- 1. Пользователь нажимает кнопку **«Регистрация»** (см. рисунок 1.3).
- 2. Открывается форма, в которую пользователь вводит:
 - адрес электронной почты;
 - пароль;
 - имя персонажа.
- 3. Система проверяет **уникальность** введённого адреса электронной почты.
- 4. Пользователь подтверждает согласие с:
 - **политикой конфиденциальности;**
 - **правилами использования системы.**
- 5. Пользователь нажимает кнопку **«Зарегистрироваться»**.
- 6. Данные отправляются на сервер.
- 7. Система создаёт **новую учётную запись**.
- 8. Пользователь получает сообщение об **успешной регистрации**.
- 9. Пользователь автоматически перенаправляется на **главную страницу** приложения.

Постусловия:

Учётная запись создана, и пользователь может авторизоваться в системе.

Альтернативные сценарии:

- **Условие 1:** Введённый адрес электронной почты уже используется.
 - Отображается сообщение с просьбой ввести другой адрес или пройти авторизацию.
 - Пользователь возвращается к форме ввода данных.
- **Условие 2:** Сервер временно недоступен.
 - Система отображает сообщение об ошибке и предлагает повторить попытку позже.

					09.02.07 7898.2025.У П.02.01	Лист
						10
Изм	Лист	№ документа	Подпись	Дата		

Прецедент 2. Авторизоваться

Цель:

Пользователь вводит свои учётные данные, чтобы получить доступ к своему аккаунту и персонализированному игровому процессу.

Предусловия:

Пользователь находится в **основном меню** приложения.

Основной сценарий:

- 1. Пользователь нажимает кнопку **«Вход»**.
- 2. Система отображает **форму авторизации**, содержащую поля для ввода:
 - адреса электронной почты;
 - пароля (см. рисунок 1.4).
- 3. Пользователь вводит необходимые данные.
- 4. Пользователь нажимает кнопку **«Войти»**.
- 5. Система проверяет корректность введённых данных и их наличие в базе данных.
- 6. При успешной проверке система предоставляет доступ к аккаунту и переводит пользователя в **основное меню**.

Постусловия:

Пользователь авторизован и может пользоваться доступными функциями, связанными с его учётной записью.

Альтернативные сценарии:

- **Условие 1:** Введены некорректные данные.
 - Система отображает сообщение об ошибке.
 - Пользователю предлагается повторить ввод данных.
- **Условие 2:** Учётная запись не существует.
 - Пользователю предлагается перейти к регистрации, нажав кнопку **«Регистрация»**.
- **Условие 3:** Пользователь забыл пароль.
 - На экране авторизации пользователь может нажать **«Забыли пароль?»**, чтобы инициировать процедуру восстановления пароля.
- **Условие 4:** Сервер недоступен.
 - Отображается сообщение о технической ошибке и предложении повторить позже.

					09.02.07 7898.2025.У П.02.01	Лист
						11
Изм	Лист	№ документа	Подпись	Дата		

Прецедент 3. Играть

Цель:

Игрок запускает игровой процесс, управляет персонажем и соревнуется с другими игроками в режиме реального времени.

Предусловия:

Игрок находится в **основном меню** приложения.

Основной сценарий:

- 1. Игрок нажимает кнопку «Играть».
- 2. Система загружает **игровую карту** и отображает её на экране (см. рисунок 1.5).
- 3. Система создаёт персонажа игрока и размещает его в стартовой точке.
- 4. Игрок управляет драконом, **собирает еду**, увеличивая его длину.
- 5. При нажатии определённой кнопки активируется **ускорение** на *n* секунд.
- 6. Если персонаж сталкивается с препятствием или другим игроком — игра завершается.

Постусловия:

- 1. Игрок завершает игровую сессию.
- 2. Отображается **окно с результатами игры** (набранные очки и статистика).
- 3. Если результат входит в *n*-лучших, он сохраняется в **таблице лидеров**.
- 4. Игрок перенаправляется обратно в **основное меню**.

Альтернативные сценарии:

- **Условие 1:** Сервер недоступен.
 - Система отображает сообщение об ошибке и предлагает повторить попытку позже.

Прецедент 4. Посмотреть таблицу лидеров

Цель:

Пользователь просматривает список лучших игроков, отсортированных по рейтингу, за определённый период времени.

Предусловия:

Пользователь находится в **основном меню** приложения.

					09.02.07 7898.2025.У П.02.01	Лист
						12
Изм	Лист	№ документа	Подпись	Дата		

Основной сценарий:

- 1. Пользователь нажимает кнопку «**Лучшие игроки**».
- 2. Система отображает страницу с **таблицей лидеров**, содержащей список игроков, отсортированный по убыванию рейтинга (см. рисунок 1.6).
- 3. В таблице указываются:
 - имя игрока;
 - его **максимальный рейтинг**.
- 4. Пользователь может выбрать **временной интервал** (например: за день, неделю, месяц), чтобы отфильтровать результаты.
- 5. Для возврата в главное меню пользователь нажимает кнопку «**Назад**».

Постусловия:

Пользователь ознакомился с результатами лучших игроков за выбранный период времени.

Альтернативные сценарии:

- **Условие 1:** Таблица лидеров пуста.
 - Система выводит сообщение: «*В таблице лидеров пока нет результатов. Начните игру, чтобы создать первые записи!*».

Прецедент 5. Изменить настройки

Цель:

Пользователь настраивает параметры своего профиля, включая имя персонажа, модель персонажа и способ управления.

Предусловия:

Пользователь **авторизован** и находится в **основном меню** приложения.

Основной сценарий:

- 1. Пользователь нажимает кнопку «**Настройки**».
- 2. Система открывает **экран настроек профиля** (см. рисунок 1.7).
- 3. Пользователь выбирает опцию «**Изменить имя**», вводит новое имя и нажимает кнопку «**Подтвердить**» (рисунок 1.8).
- 4. Пользователь выбирает опцию «**Персонаж**» и выбирает одну из доступных **2D-моделей персонажа** (рисунок 1.10).
- 5. Пользователь выбирает опцию «**Управление**» и указывает способ управления:
 - **мышь**;
 - **клавиатура**

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	13

6. Система сохраняет внесённые изменения.
7. Пользователь возвращается на главный экран, нажав кнопку **«Назад»**.

Постусловия:

Настройки профиля обновлены. Изменения применяются при следующей игровой сессии.

Альтернативные сценарии:

- **Условие 1:** Сервер недоступен.
 - Система выводит сообщение об ошибке: *«Не удалось сохранить изменения. Повторите попытку позже.»*

Прецедент 6. Узнать справочную информацию

Цель:

Пользователь получает доступ к справочным материалам, включающим правила игры, описание персонажа и способы управления.

Предусловия:

Пользователь находится в **основном меню** приложения.

Основной сценарий:

1. Пользователь нажимает кнопку **«Справка»**.
2. Система открывает **экран справочной информации** (см. рисунок 1.11).
3. На экране отображаются текстовые и/или графические материалы, содержащие:
 - правила игры;
 - описание способностей персонажа;
 - варианты управления.
4. Пользователь при необходимости прокручивает страницу **вверх или вниз** для просмотра полной информации.
5. По завершении просмотра пользователь возвращается на главный экран, нажав кнопку **«Назад»**.

Постусловия:

Пользователь ознакомился с актуальной справочной информацией и может использовать её в процессе игры.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	14

Альтернативные сценарии:

- **Условие 1:** Сервер недоступен.
 - Система отображает сообщение об ошибке: *«Не удалось загрузить справку. Повторите попытку позже.»*

Прецедент 7. Поделиться в социальной сети

Цель:

Пользователь публикует информацию о своём результате в игре, включая ссылку на приложение, в одной из доступных социальных сетей.

Предусловия:

- Пользователь находится в **основном меню** или на **экране с результатами игры**.
- У пользователя имеется **учётная запись** в социальной сети, в которую он хочет опубликовать информацию.

Основной сценарий:

1. Пользователь нажимает кнопку **«Поделиться в социальной сети»**.
2. Система открывает экран с **выбором социальной сети**.
3. Пользователь выбирает нужную платформу (например: ВКонтакте, Telegram, и т.д.).
4. Система переходит на страницу публикации в выбранной социальной сети.
5. Пользователь просматривает предварительно сформированный текст (например: *«Я набрал 5 000 очков в Snake 2D! Попробуй побить мой рекорд!»*) и, при желании, добавляет комментарий.
6. Пользователь нажимает кнопку **«Опубликовать»**.
7. Информация о результате игры публикуется на странице пользователя в выбранной социальной сети.
8. Пользователь возвращается в **основное меню** приложения.

Постусловия:

Информация об игре успешно опубликована, и пользователь может продолжить работу с приложением.

Альтернативные сценарии:

- **Условие 1:** Пользователь не авторизован в выбранной социальной сети.
 - Система отображает сообщение с просьбой **авторизоваться или зарегистрироваться** в данной социальной сети для публикации.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	15

1.2.2 Нефункциональные требования

Клиентское приложение должно соответствовать следующим **нефункциональным требованиям**, обеспечивающим стабильность, доступность и удобство использования:

- **Кроссплатформенная совместимость:**
Приложение должно корректно работать на различных устройствах и операционных системах (в рамках проекта — преимущественно Microsoft Windows), с возможностью масштабирования под мобильные платформы. Это обеспечит доступность игры для широкой аудитории.
- **Удобство использования:**
Интерфейс должен быть **интуитивно понятным**, с минимальным количеством действий для начала игры. Навигация между экранами и выполнение игровых действий должны быть простыми и логичными.
- **Производительность:**
Игра должна работать **плавно** на большинстве целевых устройств, обеспечивая стабильный FPS и минимальные задержки при сетевом взаимодействии.
- **Масштабируемость:**
Архитектура приложения должна предусматривать возможность интеграции новых модулей (например, чат, достижения, внутриигровой магазин и т.д.) без необходимости полной переработки проекта.
- **Надёжность:**
Система должна корректно обрабатывать ошибки (например, потерю соединения с сервером, отсутствие данных, сбой при авторизации) и предоставлять пользователю понятные сообщения с вариантами действий.
- **Безопасность:**
Персональные данные пользователей (адрес электронной почты, пароль) должны храниться и передаваться с использованием **безопасных протоколов** и алгоритмов шифрования (например, HTTPS, хэширование паролей).

1.3 Диаграмма потока экранов

На основе анализа пользовательских прецедентов, представленных в разделе 1.2.1, был сформирован перечень необходимых экранов (состояний) приложения и логика переходов между ними. В результате проектирования навигационной структуры была построена **диаграмма потока экранов**, отображающая взаимосвязь интерфейсных страниц и возможные действия пользователя при взаимодействии с приложением.

					09.02.07 7898.2025. У П.02.01	Лист
						16
Изм	Лист	№ документа	Подпись	Дата		

На **рисунке 1.12** представлена соответствующая диаграмма, построенная в нотации UML (диаграмма состояний). Каждый блок на диаграмме представляет отдельный экран, доступный пользователю, а стрелки обозначают переходы между этими экранами, обусловленные действиями или выбором пользователя.

В частности, с **основного меню** доступны переходы к следующим экранам:

- окно входа в учётную запись;
- регистрация нового пользователя;
- игровой процесс;
- таблица лидеров;
- окно настроек;
- справка;
- окно публикации в социальных сетях.

Каждый из экранов, за исключением игрового поля, содержит обратный переход к основному меню, обеспечивая **интуитивно понятную навигацию** и возможность легко вернуться к начальному состоянию. В случае с игрой, возврат осуществляется по завершении игровой сессии.

Диаграмма позволяет наглядно представить всю структуру приложения до начала реализации графического интерфейса, что способствует улучшению **пользовательского опыта (UX)** и **качеству архитектурного проектирования** интерфейса.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	17

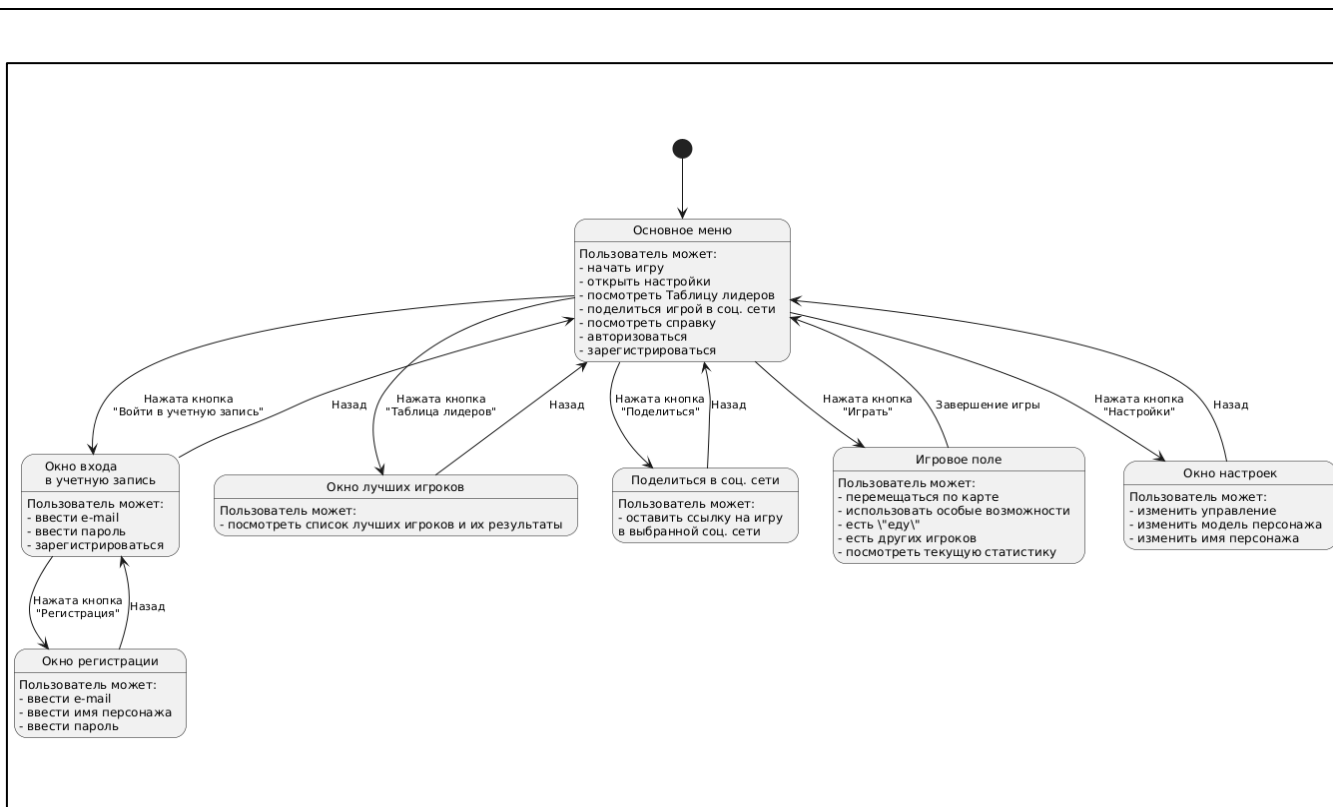


Рисунок 1.12 — Диаграмма потока экранов

1.4 Выводы по главе

В результате выполнения первого этапа разработки были достигнуты следующие результаты:

1. Проведён **анализ существующих аналогов** — популярных многопользовательских игр жанра «змея», таких как *Snake.io*, *Wormax.io* и *Slither.io*. Анализ позволил выделить успешные решения, а также определить недостатки, которые планируется устранить в рамках собственной реализации.
2. На основе анализа была составлена **спецификация функциональных требований**, оформленная в виде **диаграммы прецедентов** и детализированного текстового описания сценариев использования.
3. Также были сформулированы **нефункциональные требования**, определяющие общие характеристики клиентского приложения: удобство использования, производительность, надёжность, безопасность и масштабируемость.

					09.02.07	Лист
					7898.2025. У П.02.01	18
Изм	Лист	№ документа	Подпись	Дата		

4. Разработана **диаграмма потока экранов**, отражающая структуру пользовательского интерфейса и переходы между экранами. Это позволило определить логику взаимодействия пользователя с приложением до начала непосредственной реализации.

Таким образом, в данной главе заложены теоретические и проектные основы, необходимые для дальнейшего этапа разработки клиентской части приложения.

Глава 2. Проектирование

Архитектура разрабатываемого приложения была составлена на основе ранее сформулированной **спецификации требований** (см. рисунок 2.1). Система реализуется по **клиент-серверной модели** и состоит из двух ключевых компонентов:

- **Клиентского приложения** (реализованного в Unity);
- **Серверного приложения**, обеспечивающего хранение и обработку игровых и пользовательских данных.

Архитектура взаимодействия

Для авторизации и получения информации о пользователе клиентское приложение взаимодействует с сервером по протоколу **TCP**. В ответ сервер возвращает необходимые данные, хранящиеся в базе данных, включающей:

- имя пользователя;
- адрес электронной почты;
- пароль (в зашифрованном виде);
- лучший результат в игре.

Во время самой игровой сессии применяется **сетевой протокол UDP**, с использованием **Unity Transport**. Это позволяет минимизировать задержки при передаче данных между клиентами и сервером. Сервер при этом:

- хранит информацию о подключённых клиентах;
- обрабатывает игровые события, поступающие от клиентов;
- выполняет широковещательную рассылку состояния игры другим участникам сессии.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	19

Компоненты клиентского приложения

Клиентская часть приложения включает следующие элементы:

- **Экран основного меню**, содержащий доступ к подменю:
 - форма авторизации;
 - таблица лидеров;
 - справочная информация;
 - настройки.
- **Игровой экран**, отображающий основную 2D-сцену с управляемыми и сетевыми объектами.
- **Сетевые объекты**, такие как персонажи игроков и еда, синхронизируемые между клиентами.
- **Файл настроек** (формата .ini), в котором сохраняются пользовательские параметры:
 - способ управления (клавиатура/мышь);
 - выбранная модель персонажа и имя.

При нажатии кнопки «Играть», пользователь переходит на игровой экран, который инициализирует работу с сетевыми объектами и подключение к серверу.

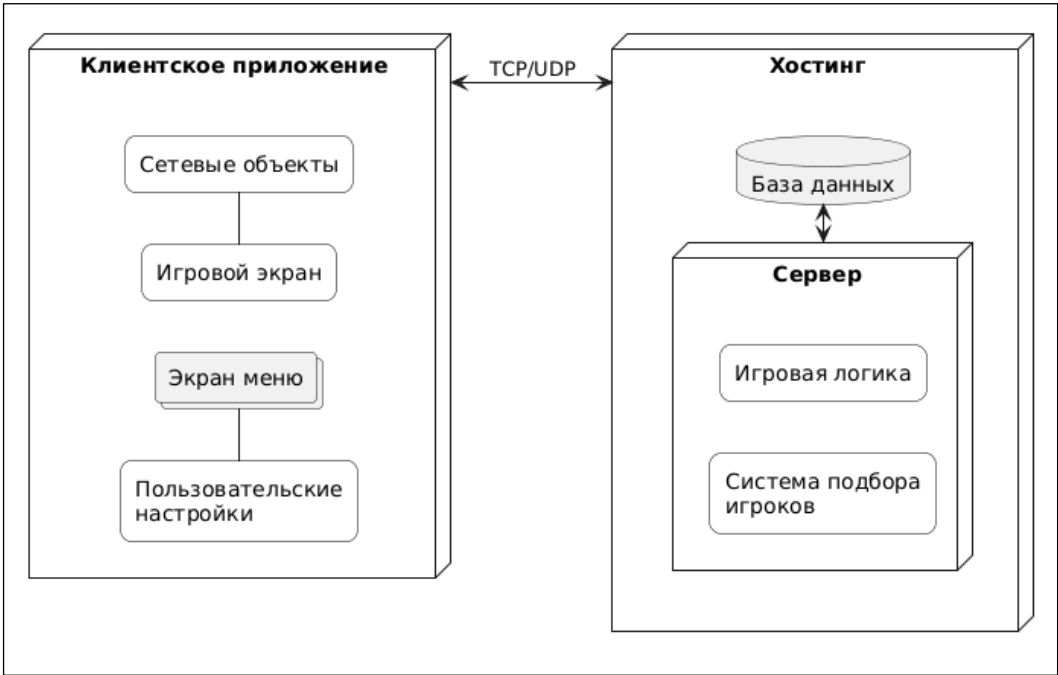


Рисунок 2.1 — Архитектура приложения

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	20

Методология проектирования

Для обеспечения соответствия требованиям и наглядного проектирования была выбрана методология **ICONIX** [6], ориентированная на использование **прецедентов** как основы проектного анализа. ICONIX позволяет выстроить логичную и последовательную модель разработки, начиная с use-case'ов и заканчивая реализацией классов.

В процессе проектирования были выделены ключевые операции, применяемые к нескольким сущностям приложения:

- подключение к игре;
- управление персонажем;
- обработка игровых событий;
- визуализация;
- синхронизация и обновление состояния игры.

Для описания этих операций были разработаны:

- **диаграммы пригодности;**
- **диаграммы последовательности.**

На их основе были сформированы **диаграммы классов**, используемые в процессе реализации клиентской логики и сетевых компонентов.

2.1 Диаграммы пригодности

2.1.1 Прецедент «Играть»

Данный прецедент описывает основную пользовательскую цель — участие в игровой сессии.

Пользователь запускает приложение и нажимает кнопку **«Играть»** в главном меню. После этого клиентское приложение инициирует **сетевое соединение с сервером**, отправляя запрос на подключение. При успешном подключении пользователь попадает в текущую игровую сессию, где:

- загружается **игровая сцена (мир)**;
- создаётся и отображается **персонаж пользователя**;
- запускается обмен данными с другими клиентами через сервер.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	21

Пользователь получает возможность управлять своим персонажем, собирать игровые объекты (еду), использовать особые возможности, а также **взаимодействовать с другими игроками в реальном времени**.

В случае, если соединение с сервером установить не удалось (например, сервер недоступен, превышено время ожидания, или возникли сетевые ошибки), пользователю отображается **сообщение об ошибке сети**, и повторный вход в игровую сессию становится возможным только после устранения проблемы.

На **рисунке 2.2** представлена **диаграмма пригодности** (use case realization), иллюстрирующая ключевые действия и взаимосвязи между участниками и компонентами системы в рамках данного прецедента.

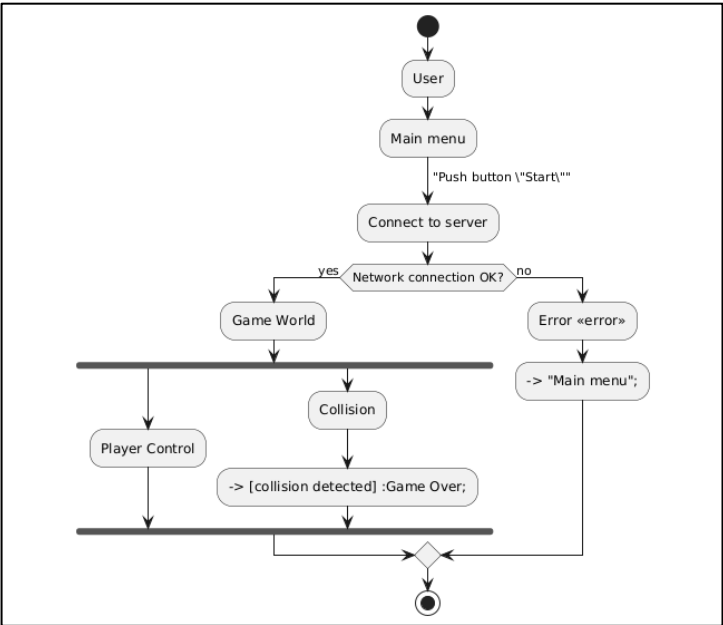


Рисунок 2.2 — Диаграмма пригодности прецедента «Играть»

Примечание:

Представленная ниже диаграмма отражает **целевую структуру прецедента**, спроектированную на этапе подготовки архитектуры. Некоторые элементы, такие как **обработка ошибок соединения и отображение сообщений**, на момент завершения практики не реализованы, но предусмотрены как часть будущего расширения функционала.

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		22

2.1.2 Прецедент «Авторизоваться»

Пользователь запускает приложение и нажимает кнопку **«Войти»** в основном меню. После этого открывается **форма авторизации**, где он вводит:

- адрес электронной почты;
- пароль;
- имя пользователя (при необходимости).

Введённые данные передаются на **сервер**, где происходит проверка их корректности. В зависимости от результата в клиентском приложении отображается:

- сообщение об **успешной авторизации**, после чего пользователь получает доступ к функционалу;
- или сообщение об **ошибке** (например, неверный пароль, отсутствие учётной записи и т.д.).

Если у пользователя **отсутствует учётная запись**, он переходит к **регистрации** — заполняет форму с теми же данными (адрес электронной почты, имя персонажа, пароль). На сервер отправляется **запрос на создание аккаунта**, и в ответ клиент получает сообщение:

- об успешной регистрации;
- либо об ошибке (например, такой e-mail уже используется).

На **рисунке 2.3** представлена **диаграмма пригодности**, иллюстрирующая процесс авторизации и регистрации, включая взаимодействие клиента с сервером и возможные альтернативные сценарии.

					09.02.07 7898.2025.У П.02.01	Лист
						23
Изм	Лист	№ документа	Подпись	Дата		

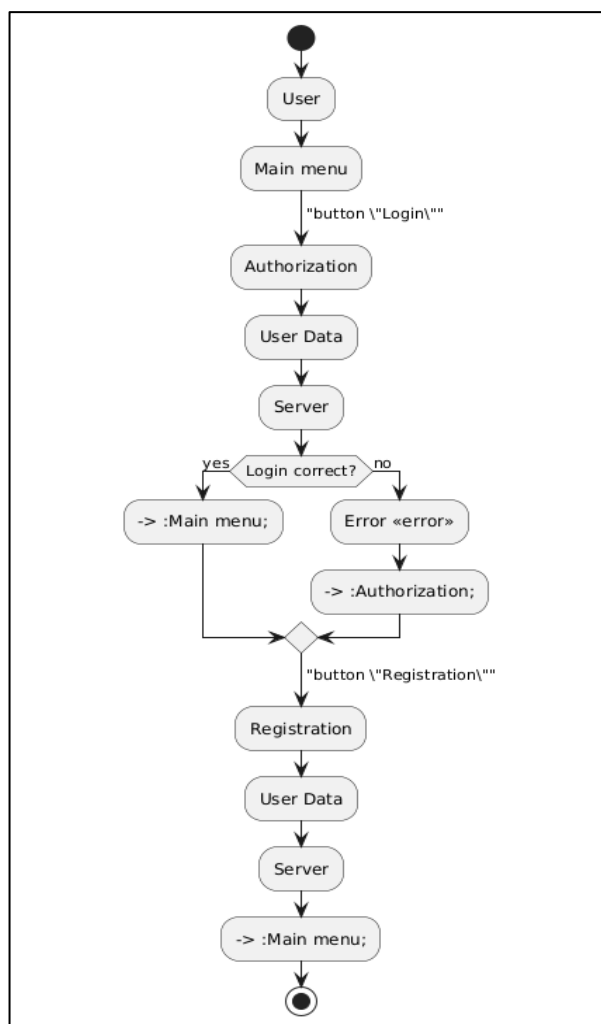


Рисунок 2.3 — Диаграмма пригодности прецедента «Авторизоваться»

2.1.3 Прецедент «Изменить настройки»

Пользователь переходит в раздел **настроек** из основного меню приложения. В данном разделе доступны следующие функции:

- изменение **способа управления** (например, клавиатура или мышь);
- выбор **модели персонажа** из доступных вариантов;
- изменение **имени персонажа**.

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		24

Каждое изменение сохраняется локально в **файле конфигурации (формат .ini)**, который используется клиентским приложением при запуске игровых сессий. Это обеспечивает сохранность пользовательских предпочтений между запусками приложения.

На **рисунке 2.4** представлена **диаграмма пригодности**, демонстрирующая процесс перехода пользователя в меню настроек, выбор параметров и возврат в основное меню.

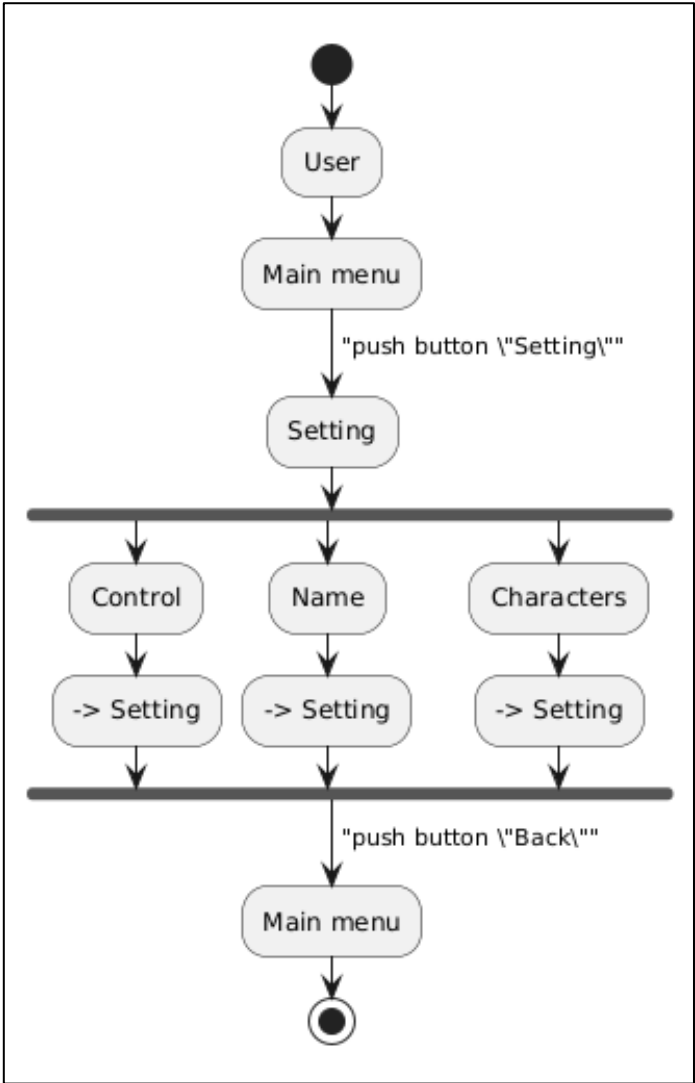


Рисунок 2.4 — Диаграмма пригодности прецедента «Изменить настройки»

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	25

2.2 Диаграммы последовательности

Значительная часть логики системы сосредоточена на взаимодействии компонентов во время работы **основного меню** и **игрового поля**. Для визуализации процессов, происходящих между клиентом и сервером, в данном разделе представлены **диаграммы последовательностей** (sequence diagrams), отражающие ключевые сценарии работы системы.

На **рисунке 2.5** изображена диаграмма последовательности для прецедента «Играть», демонстрирующая процесс взаимодействия между клиентским приложением и серверной частью.

В рамках данного сценария:

- клиент отправляет запрос на подключение к игровой сессии;
- сервер подтверждает соединение и передаёт начальные данные о состоянии мира;
- в процессе игры сервер пересылает клиенту:
 - актуальные **позиции объектов еды**;
 - текущую **позицию и длину** управляемого персонажа;
 - информацию о других игроках (по необходимости);
- после завершения игровой сессии клиент получает и отображает **итоговый результат**, включая количество очков, место в рейтинге и другие параметры.

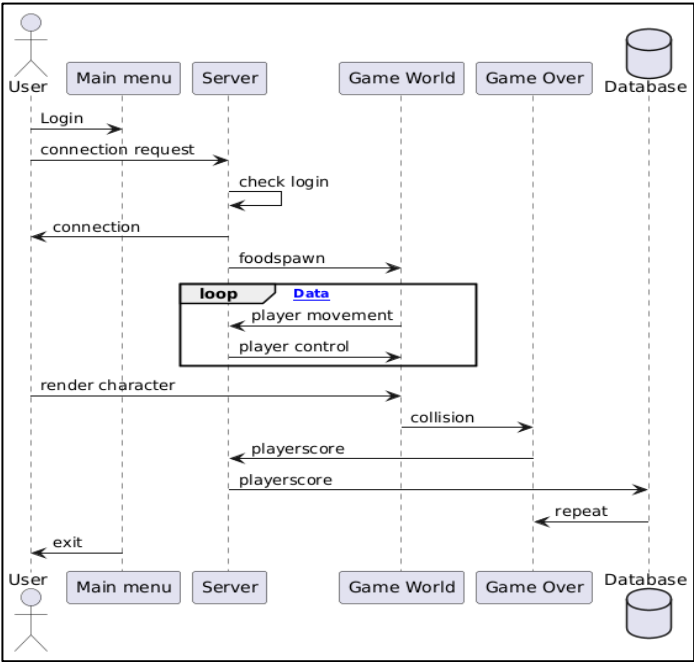


Рисунок 2.5 Диаграмма последовательности «Играть»

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	26

На рисунке 2.6 представлена диаграмма последовательности, иллюстрирующая процесс авторизации пользователя в системе. Диаграмма также охватывает альтернативный сценарий — регистрацию нового пользователя в случае отсутствия учётной записи.

В рамках данного сценария реализуются следующие этапы:

1. Пользователь нажимает кнопку «Войти» в главном меню и вводит адрес электронной почты и пароль.
2. Клиентское приложение формирует **запрос на авторизацию** и отправляет его на сервер.
3. Сервер, в свою очередь, выполняет **проверку данных** через базу данных.
4. В зависимости от результата, клиент получает ответ:
 - об **успешной авторизации** — и получает доступ к профилю;
 - или об **ошибке входа** — с предложением повторить попытку или зарегистрироваться.
5. Если пользователь не имеет аккаунта, он может перейти к **регистрации**.
6. Клиент отправляет регистрационные данные (имя персонажа, e-mail, пароль) на сервер.
7. Сервер сохраняет учётную запись в базе данных и возвращает результат регистрации клиенту.

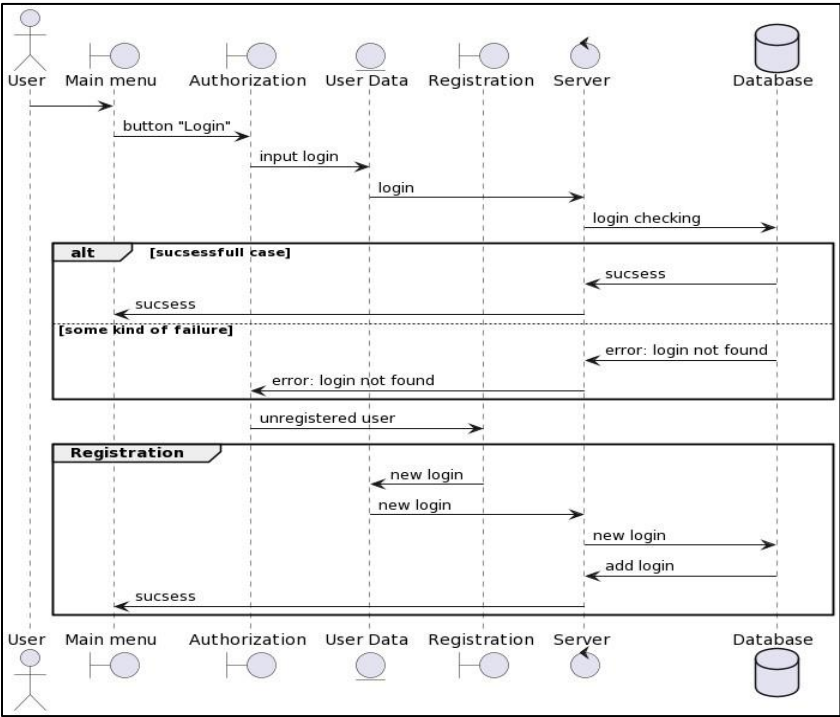


Рисунок 2.6. Диаграмма последовательности «Авторизация»

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	27

2.2.3 Прецедент «Изменить настройки»

На рисунке 2.7 представлена диаграмма последовательности, отражающая процесс изменения пользовательских настроек в клиентском приложении.

Пользователь, находясь в основном меню, переходит в **раздел настроек**, где ему доступны следующие действия:

- изменение **способа управления** (например, переключение между мышью и клавиатурой);
- изменение **имени персонажа**;
- выбор новой **модели персонажа** из доступных 2D-вариантов.

Все изменения, выполненные пользователем, **обрабатываются на стороне клиента** и записываются в **локальный конфигурационный файл** (формата `.ini`). Эти данные используются при запуске игры и могут быть загружены автоматически при следующем входе в приложение.

Поскольку операции не требуют обращения к серверу, все действия происходят в рамках локального взаимодействия между компонентами клиентского приложения:

- пользовательский интерфейс;
- система управления настройками;
- модуль сохранения данных.

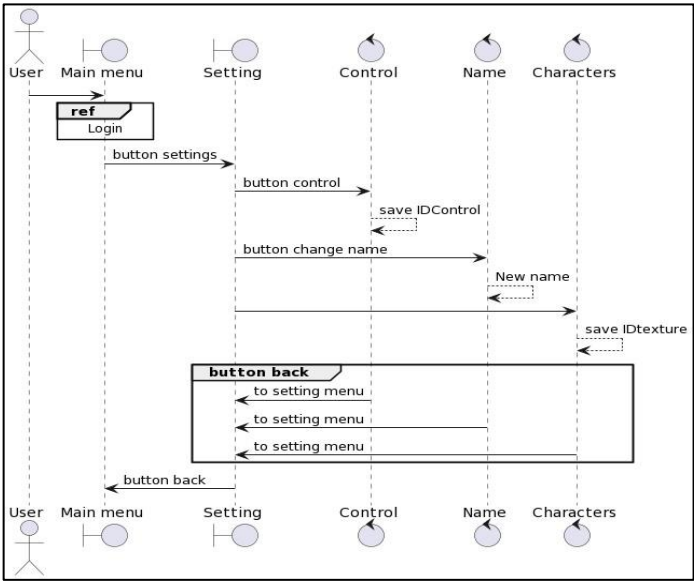


Рисунок 2.7 — Диаграмма последовательности «Изменить настройки»

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	28

2.3 Диаграммы классов

В рамках реализации игровой логики все игроки обладают **одинаковым набором параметров**, таких как:

- положение головы;
- положение хвоста;
- длина хвоста;
- скорость передвижения;
- текущее количество очков.

Для упрощения и унификации реализации была спроектирована **иерархия классов**, в которой ключевым элементом является базовый класс `Player`. Все игровые персонажи наследуются от этого класса.

Внутри `Player` реализуется **валидация данных**, предотвращающая внешнюю подмену критически важных параметров (например, скорости или длины). Методы класса взаимодействуют с сервером и принимают только корректные данные, прошедшие проверку.

Основные компоненты:

- **PlayerControl**
Отвечает за передачу на сервер координат положения указателя мыши. Сервер, в свою очередь, интерпретирует это как направление движения головы персонажа.
- **CameraController**
Выполняет слежение за игровым персонажем и динамически позиционирует камеру.
- **Collisions**
Обрабатывает события столкновений с едой. В случае соприкосновения вызывается обновление длины персонажа.
- **PlayerLength**
Отвечает за добавление новых сегментов хвоста при увеличении длины, вызывается сервером после события столкновения.
- **Tail**
Управляет движением хвоста — сегменты перемещаются вслед за головой, повторяя её траекторию.
- **PlayerStats**
Хранит и отображает текущие игровые показатели: количество очков, позицию игрока в сессии и другие метрики.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	29

На рисунке 2.8 представлена диаграмма классов, иллюстрирующая архитектуру объектов, связанных с реализацией поведения игрока.

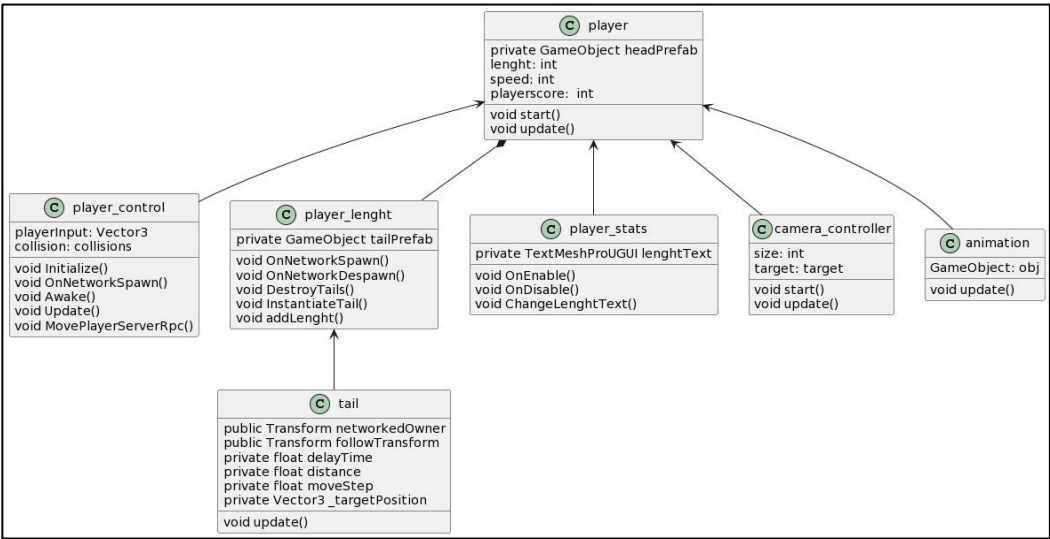


Рисунок 2.8 — Диаграмма классов архитектуры игрока

Диаграмма классов: появление объектов на сцене

Для появления объектов на игровом поле — таких как **игроки** и **единицы еды** — используется специализированный сетевой компонент. Связь между клиентом и сервером осуществляется посредством класса **NetworkManager**.

Класс **NetworkManager** отвечает за:

- получение от сервера данных о количестве объектов еды;
- определение координат их появления;
- получение информации о позиции появления новых игроков;
- инициализацию игровых объектов на сцене на основе полученных данных.

Таким образом, **NetworkManager** выступает в роли **сетевого координатора**, синхронизирующего сцену между всеми участниками сессии и обеспечивающего актуальное состояние игрового мира.

					09.02.07	Лист
					7898.2025. У П.02.01	30
Изм	Лист	№ документа	Подпись	Дата		

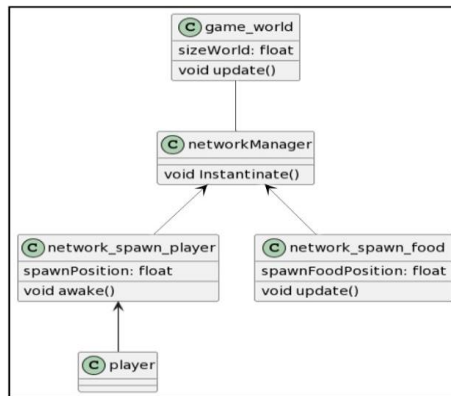


Рисунок 2.9 — Диаграмма классов для процесса появления объектов на игровой сцене

Диаграмма классов: процесс авторизации

Класс **AuthorizationController** реализует процесс аутентификации пользователя в клиентском приложении. Он принимает данные, введённые пользователем (адрес электронной почты, пароль), и передаёт их на игровой сервер для проверки.

Основная логика работы:

- При корректных данных от сервера поступает положительный ответ, и пользователь автоматически **перенаправляется в главное меню**.
- В случае ошибки (например, неверный логин или пароль) на экран выводится **сообщение об ошибке авторизации**, и пользователь остаётся в **меню авторизации**.
- Если у пользователя нет учётной записи, он может перейти в **меню регистрации**, где происходит повторный ввод данных и отправка на сервер для создания новой записи.

Класс **AuthorizationController** взаимодействует с:

- **UI_LoginForm** — интерфейсная форма ввода данных;
- **ServerConnection** — компонент, обеспечивающий сетевое взаимодействие с сервером;
- **UserModel** — объект, хранящий введённые пользователем данные;
- **NavigationManager** — логика переключения между экранами (авторизация, регистрация, главное меню);
- **ErrorHandler** — компонент, обрабатывающий ошибки и выводящий сообщения пользователю.

					09.02.07 7898.2025. У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		31

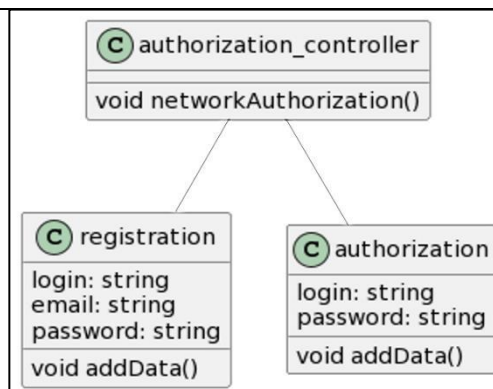


Рисунок 2.10 — Диаграмма классов для процесса авторизации

2.4 Диаграмма базы данных

Для хранения пользовательских данных на стороне сервера используется база данных, структура которой позволяет надёжно сохранять основную информацию, необходимую для авторизации и ведения статистики.

База данных содержит следующую информацию:

- **идентификатор пользователя** (уникальный ключ);
- **лучший игровой результат** (максимальное количество очков, набранных за сессию);
- **регистрационные данные**, включая:
 - логин (отображаемое имя в игре);
 - адрес электронной почты;
 - пароль (в зашифрованном виде, например, с использованием хэширования).

Такое построение таблицы позволяет:

- реализовать корректную авторизацию и регистрацию;
- хранить игровую статистику;
- при необходимости — расширить структуру (например, добавить дату последнего входа, аватар, достижения и т.п.).

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		32

На рисунке 2.11 представлена диаграмма базы данных, отражающая структуру таблицы пользователей и связи между сущностями.



Рисунок 2.11 — ER-диаграмма базы данных

2.5 Выводы по главе

В рамках проектирования клиентской части сетевой игры, на основе ранее сформированной спецификации требований, были выполнены следующие действия:

1. Разработаны **диаграммы пригодности** для ключевых пользовательских прецедентов, таких как: «Играть», «Авторизоваться», «Изменить настройки». Эти диаграммы стали основой для построения соответствующих **диаграмм последовательности**, которые отразили взаимодействие между компонентами системы на более детальном уровне.
2. На основе диаграмм поведения и сценариев использования была разработана **диаграмма классов**, описывающая архитектуру игровых сущностей и их взаимодействие внутри клиентского приложения.
3. Предложена общая **архитектура системы**, отражающая клиент-серверное взаимодействие, включая механизм передачи данных во время игры и при авторизации.
4. Построена **структура базы данных**, предназначенная для хранения пользовательской информации и игровой статистики. Эта структура обеспечивает безопасную авторизацию и возможность масштабирования функциональности в дальнейшем.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	33

Глава 3. Реализация и тестирование

В процессе учебной практики реализация проекта HEURO COIL велась на основе заранее подготовленного технического задания, приведённого ниже.

Некоторые положения ТЗ (например, использование библиотеки Netcode for GameObjects и интеграция таблицы лидеров) были адаптированы в процессе работы: для обеспечения более быстрой и гибкой реализации была выбрана альтернативная сетевая библиотека Mirror, а дополнительные модули проектировались, но не были реализованы в рамках ограниченного времени.

Несмотря на это, основные цели практики были достигнуты — разработана локальная и сетевая версия игры, настроена архитектура, реализован пользовательский интерфейс, синхронизация и взаимодействие объектов в реальном времени.

Техническое задание для проекта HEURO COIL

1. Общие сведения

Название проекта:
HEURO COIL

Тип проекта:
Многопользовательская 2D-игра в жанре "змейка".

Платформа разработки:
Unity 2022.3.61f1 (или выше).

Целевая платформа:
Microsoft Windows с возможностью дальнейшей масштабируемости на мобильные устройства.

2. Цель проекта

Разработка многопользовательской 2D-игры "HEURO COIL", сочетающей элементы классической игры "Змейка" с современным онлайн-соревнованием между пользователями, возможностью персонализации персонажей и реализацией

					09.02.07	Лист
					7898.2025. У П.02.01	34
Изм	Лист	№ документа	Подпись	Дата		

3. Требования к проекту

3.1 Функциональные требования

- Реализация сетевого мультиплеера на базе Netcode for GameObjects.
- Возможность локальной оффлайн-игры для тестирования основных механик.
- Управление персонажем (змейкой) в реальном времени.
- Сбор объектов (еды) с целью увеличения длины персонажа.
- Автоматическое создание объектов еды с выбором префаба из заданного списка.
- Отображение таблицы лидеров для текущей игровой сессии.
- Завершение игры при столкновении с другим игроком или границей карты.
- Возможность перезапуска игровой сессии без выхода из приложения.
- Основное меню с доступом к следующим разделам:
 - Запуск игры;
 - Просмотр таблицы лидеров;
 - Настройки профиля;
 - Справочная информация;
 - Выход из игры.

3.2 Нефункциональные требования

- Обеспечение высокой производительности и минимальной задержки сетевого взаимодействия.
- Совместимость с операционной системой Microsoft Windows.
- Использование Unity в качестве основного игрового движка.
- Масштабируемость архитектуры для возможной интеграции дополнительных модулей (авторизация, чат, магазин, достижения).
- Обеспечение надёжности обработки ошибок (потеря соединения, ошибки при авторизации и пр.).
- Безопасная передача данных при наличии учётных записей.

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		35

4. Структура экранов и навигация

Приложение должно содержать следующие экраны:

- Главное меню;
- Вход в учётную запись (опционально);
- Регистрация нового пользователя (опционально);
- Игровой процесс;
- Таблица лидеров;
- Настройки персонажа;
- Справочная информация;
- Экран публикации результата в социальной сети (опционально).

Переходы между экранами должны быть интуитивно понятными, с возможностью возврата в главное меню на большинстве экранов.

5. Особенности реализации

- Начальный этап проекта предусматривает разработку базовой версии игры с локальным режимом, без подключения к серверу.
- Сетевая функциональность будет внедряться на последующих этапах путём интеграции RPC-методов (ServerRpc, ClientRpc).
- Логика игры и сетевая логика должны быть отделены друг от друга для облегчения поддержки и масштабируемости.
- Первоначально будет использоваться базовая 2D-графика, с возможностью последующей модернизации визуального оформления.
- Интерфейс игры будет ориентирован на простоту и удобство пользователя, с последующей доработкой визуальных эффектов.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	36

6. Разработка и анализ требований к программной среде

6.1 Средства разработки

Для реализации проекта используется игровой движок **Unity**, обеспечивающий поддержку 2D-графики, мультиплатформенной сборки и интеграцию сетевых библиотек.

Средства разработки:

- **Unity 2022.3.61f1 (LTS)** — основной движок для разработки игры;
- **Visual Studio 2022** с установленной поддержкой Unity и .NET;
- **Netcode for GameObjects** — библиотека для реализации сетевого взаимодействия;
- **Unity Transport** — библиотека для обеспечения сетевого трафика;
- **TextMeshPro** — для отображения текста в интерфейсе игры;
- **Git** — для управления версиями проекта.

Дополнительно:

- Плагины Asset Store могут использоваться для улучшения визуальных эффектов.

6.2 Целевая платформа

Основная платформа запуска:

- **Операционная система Microsoft Windows 10/11.**

Потенциальные дополнительные платформы:

- Android (рассматривается в перспективе при дальнейшем развитии проекта).

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	37

6.3 Требования к системам разработки и запуска

Минимальные требования для запуска проекта в среде разработки:

- Процессор: Intel Core i5 (6-го поколения) или аналогичный AMD;
- Оперативная память: 8 ГБ и более;
- Видеокарта: совместимая с DirectX 11;
- Свободное место на диске: минимум 2 ГБ.

Минимальные требования для конечного пользователя:

- Операционная система: Windows 10 (64-bit);
- Процессор: Intel Core i3 или аналогичный AMD;
- Оперативная память: 4 ГБ;
- Видеокарта: встроенная графика уровня Intel HD 4000 или выше;
- Свободное место: 500 МБ.

6.4 Ограничения и особенности программной среды

- Использование сетевой библиотеки Netcode for GameObjects требует подключения к интернету для полноценного мультиплеерного режима.
- Базовая графика реализуется в формате 2D с оптимизацией под устройства средней производительности.
- Возможное расширение проекта до мобильной платформы требует дополнительных адаптаций интерфейса и оптимизации производительности.

					09.02.07 7898.2025.У П.02.01	Лист
						38
Изм	Лист	№ документа	Подпись	Дата		

3.1 Выбор инструментов

Для реализации клиент-серверного приложения была выбрана игровая платформа **Unity** [7], поскольку она поддерживает мультиплатформенную разработку, включая операционные системы:

- Windows
- macOS
- Linux
- iOS
- Android и другие

Это позволяет создавать приложения, которые работают на различных устройствах, обеспечивая гибкость и доступность для пользователей.

Unity предоставляет мощный игровой движок, включающий:

- инструменты визуального редактирования сцен;
- удобную интеграцию пользовательского интерфейса (UI);
- систему событий, анимаций и ввода;
- обширные возможности для реализации логики и взаимодействия с 2D-объектами.

Выбор сетевой библиотеки: от NGO к Mirror

В рамках первоначального плана проекта для реализации сетевой логики была выбрана библиотека **Netcode for GameObjects (NGO)** — официальное решение от Unity, предоставляющее высокоуровневый API для построения многопользовательских игр.

На раннем этапе разработки была выполнена начальная настройка проекта под NGO: установлены необходимые пакеты, добавлены `NetworkObject`, `ServerRpc/ClientRpc`, реализованы базовые вызовы для управления игроками.

Однако на практике возникли ограничения, связанные с архитектурной сложностью и недостаточной гибкостью библиотеки в учебных условиях. Кроме того, NGO требует строгой конфигурации префабов и времени на освоение дополнительных концепций (`NetworkBehaviour`, `NetworkManager`, `Scene Management`), что замедляло процесс разработки и отвлекало от основной задачи — интеграции сетевого модуля в игру.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	39

В результате было принято **осознанное решение** заменить **NGO** на библиотеку **Mirror**, которая имеет схожую концепцию UNet, доступную документацию и лучше подходит для быстрой сборки прототипов. Mirror позволяет быстрее реализовать:

- подключение и синхронизацию игроков (Host, Client, Server);
- передачу данных между клиентами и сервером через RPC;
- сетевой спавн объектов (еды, хвоста, игроков);
- поддержку простого NetworkManager с подключаемыми префабами.

Этот переход позволил сосредоточиться на **практической реализации логики хоста и клиента**, а также обеспечить полноценную демонстрацию многопользовательского взаимодействия в рамках учебной практики.

4. Входные и выходные данные

Разработка многопользовательской игры NEURO COIL предполагает обработку различных типов входных и выходных данных как на клиентской, так и на серверной стороне. Ниже приведено их подробное описание с привязкой к реализованной логике.

4.1 Входные данные

Тип данных	Источник	Применение
Действия пользователя	Клавиатура, мышь, UI	Нажатие клавиш (движение персонажа); Щелчки по кнопкам (Host, Client, Exit); Нажатие ESC
Позиция курсора	Мышь (Input.mousePosition)	Определение направления движения игрока; Поворот головы змейки
Настройки звука	Слайдеры UI (Slider.value)	Уровни Master, Music, SFX; сохранение в PlayerPrefs; применение через AudioManager
Сетевые события	Действия других игроков, подключения	Подключение к серверу; RPC: CmdMove, CmdAddTail; Рассылка: RpcAddTail, RpcSpawnFood
Состояние объектов	Префабы, коллайдеры, позиции	Генерация еды и хвостов; Проверка столкновений (еда, игрок, граница)

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	40

В результате было принято **осознанное решение** заменить **NGO** на библиотеку [Mirror](#), которая имеет схожую концепцию UNet, доступную документацию и лучше подходит для быстрой сборки прототипов. Mirror позволяет быстрее реализовать:

- подключение и синхронизацию игроков (Host, Client, Server);
- передачу данных между клиентами и сервером через RPC;
- сетевой спавн объектов (еды, хвоста, игроков);
- поддержку простого NetworkManager с подключаемыми префабами.

Этот переход позволил сосредоточиться на **практической реализации логики хоста и клиента**, а также обеспечить полноценную демонстрацию многопользовательского взаимодействия в рамках учебной практики.

4. Входные и выходные данные

Разработка многопользовательской игры NEURO COIL предполагает обработку различных типов входных и выходных данных как на клиентской, так и на серверной стороне. Ниже приведено их подробное описание с привязкой к реализованной логике.

4.1 Входные данные

Тип данных	Источник	Применение
Действия пользователя	Клавиатура, мышь, UI	Нажатие клавиш (движение персонажа); Щелчки по кнопкам (Host, Client, Exit); Нажатие ESC
Позиция курсора	Мышь (Input.mousePosition)	Определение направления движения игрока; Поворот головы змейки
Настройки звука	Слайдеры UI (Slider.value)	Уровни Master, Music, SFX; сохранение в PlayerPrefs; применение через AudioManager
Сетевые события	Действия других игроков, подключения	Подключение к серверу; RPC: CmdMove, CmdAddTail; Рассылка: RpcAddTail, RpcSpawnFood
Состояние объектов	Префабы, коллайдеры, позиции	Генерация еды и хвостов; Проверка столкновений (еда, игрок, граница)

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	41

4.2 Выходные данные

Тип данных	Получатель	Пример результата
Обновление визуального состояния	Игровая сцена	Движение змейки и хвоста; Изменение размера; Спавн/удаление еды
Сетевые обновления	Все клиенты	Синхронизация позиций игроков; хвостов, еды; уведомление о победе
Интерфейс	UI (Canvas)	Скрытие/появление панелей; обновление UI; финальный экран
Аудио и эффекты	AudioMixer, эффекты	Звуки поедания, фоновая музыка; громкость по слайдерам; эффекты частиц

5. Реализация

Реализация проекта NEURO COIL была разделена на несколько функциональных модулей: меню и пользовательский интерфейс, управление персонажем, генерация еды и хвоста, логика сетевого взаимодействия и система завершения игры.

5.1 Меню и интерфейс пользователя

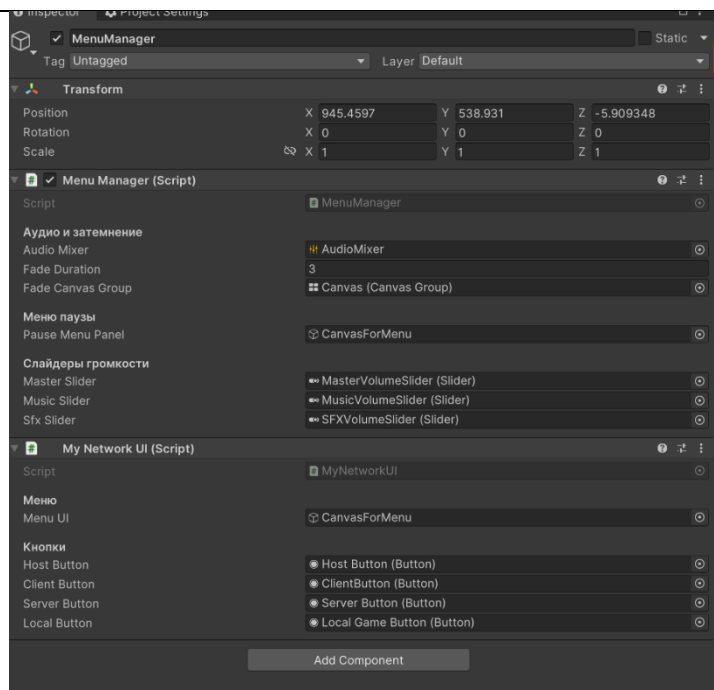
Управление интерфейсом реализовано через класс `MenuManager`, отвечающий за:

- загрузку и применение настроек громкости (Master, Music, SFX) через `AudioMixer`;
- работу панели паузы (`pauseMenuPanel`) по нажатию `ESC`;
- эффекты затемнения (`CanvasGroup`) при запуске и выходе из игры;
- сохранение значений слайдеров в `PlayerPrefs`.

Класс `MyNetworkUI` реализует базовый механизм переключения режимов:

- запуск **Host, Client, Server, Local**;
- отключение стартового меню при старте игры.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	42

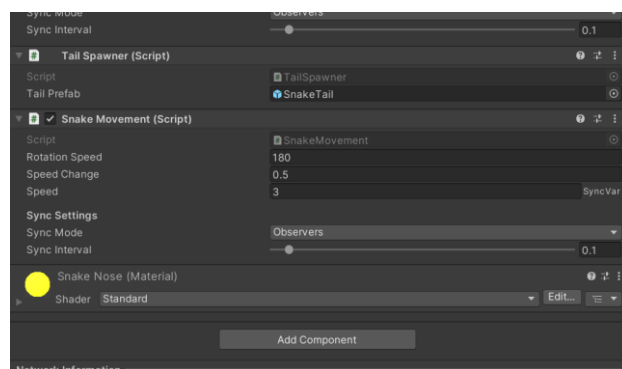


5.2 Управление змейкой

Класс `SnakeMovement` отвечает за перемещение игрока по направлению вперёд (`Vector3.forward`) и поворот с помощью горизонтального ввода (`Input.GetAxis("Horizontal")`). Компонент реализует:

- серверную реакцию на поедание еды (увеличение скорости);
- клиентскую обработку движения и поворота.

Движение происходит в **Update()** только для **авторитетного игрока**.



					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	43

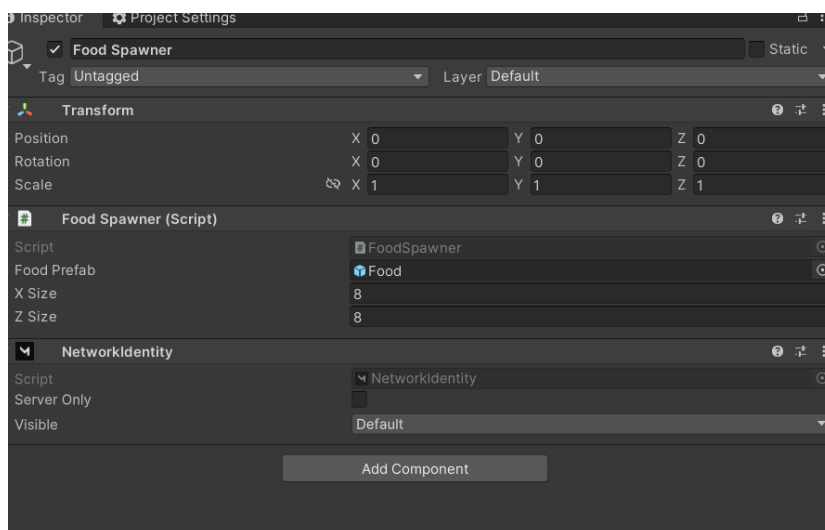
5.3 Генерация еды

Класс `FoodSpawner` размещает объект еды случайным образом в пределах указанной зоны (границы по X и Z). Сервер слушает событие `ServerOnFoodEaten`, и при его активации:

- уничтожает старую еду;
- создаёт новую с помощью `NetworkServer.Spawn()`.

Класс `Food` обрабатывает `OnTriggerEnter`, определяя столкновение с объектом с тегом "Player":

- вызывает уничтожение еды;
- запускает визуальный эффект (если задан `particlePrefab`);
- генерирует событие `ServerOnFoodEaten`.



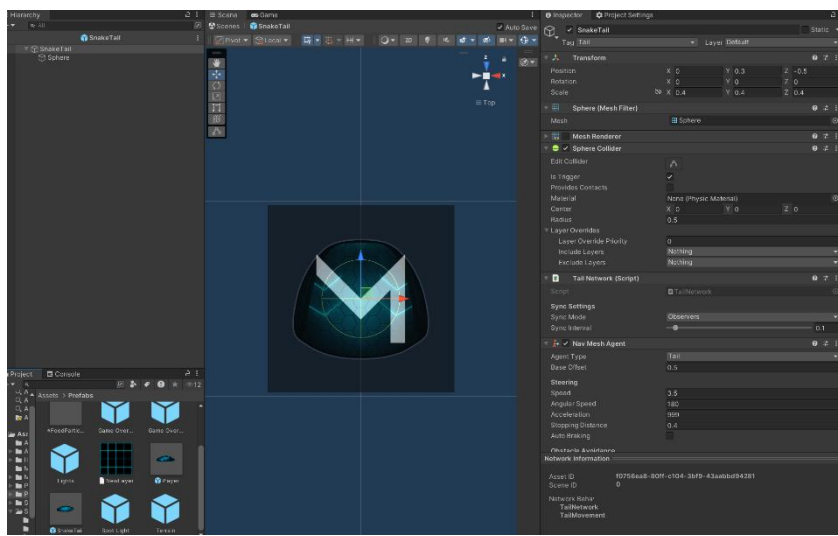
					09.02.07 7898.2025.У П.02.01	Лист
						44
Изм	Лист	№ документа	Подпись	Дата		

5.4 Хвост змейки

Хвост реализован через несколько связанных компонентов:

- **TailSpawner:** слушает событие `ServerOnFoodEaten`, создаёт новые хвостовые сегменты с `NetworkServer.Spawn` и добавляет их в список `Tails`.
- **TailNetwork:** на сервере определяет, за каким объектом (голова или последний сегмент) должен следовать данный хвост. Сохраняет ссылки на `Owner` и `Target`.
- **TailMovement:** использует `NavMeshAgent` для плавного следования за `Target`.

Такая архитектура обеспечивает естественное движение хвоста, сращивание сегментов и гибкую серверную логику.



5.5 Сетевой менеджер и запуск

Класс `SnakeNetworkManager` (наследует `NetworkManager`) переопределяет:

- `OnStartServer()` — создаёт `GameOverHandler`;
- `OnServerAddPlayer()` — вызывает базовый метод и спавнит `FoodSpawner`.

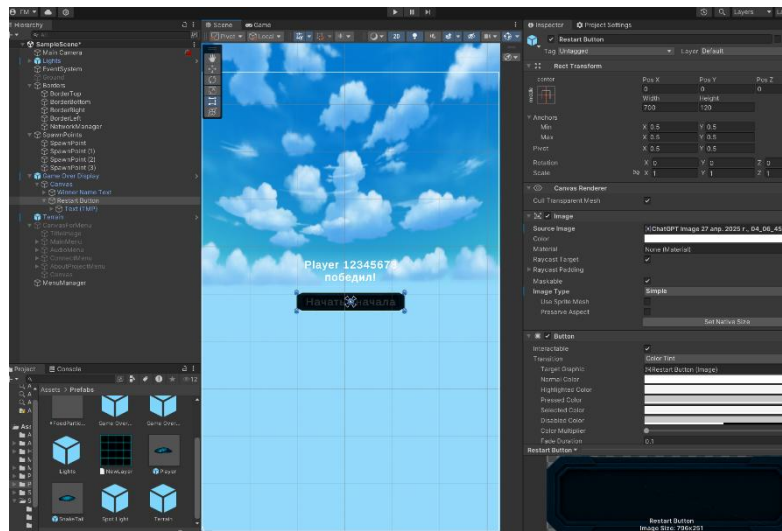
Все используемые префабы регистрируются в `Spawnable Prefabs` инспектора `NetworkManager`.

					09.02.07 7898.2025.У П.02.01	Лист
Изм	Лист	№ документа	Подпись	Дата		45

5.6 Завершение игры

Система победы реализована через:

- `GameOverHandler`: слушает события подключения/удаления игроков (`PlayerSnake`), и при достижении одного оставшегося игрока — вызывает `RpcGameOver`.
- `GameOverDisplay`: отображает победителя на UI (`TMP_Text`) и показывает `Canvas`. Также реализована возможность перезапуска (`StopHost()` / `StopClient()`).



5.7 Дополнительные компоненты

- `PlayerCameraController` — включает локальную камеру только у игрока с authority (`cam.SetActive(true)`);
- `PlayerName` — генерирует отображаемое имя игрока на сервере и синхронизирует его с клиентами через `SyncVar` и `hook`.

					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	46

6. Тестирование

Тестирование проекта NEURO COIL проводилось на уровне интеграции, с целью проверки корректного взаимодействия компонентов в рамках мультиплеерной логики и пользовательского интерфейса. Проверка осуществлялась в редакторе Unity и в собранной сборке под Windows.

№	Сценарий	Результат
1	Запуск локальной игры (режим Local)	игрок управляется, еда появляется, хвост растёт
2	Запуск Host (сервер + клиент)	локальный игрок синхронизирован, всё работает стабильно
3	Подключение второго клиента к хосту	второй игрок появляется, работает движение и еда
4	Обработка столкновений с едой	еда исчезает, хвост увеличивается
5	Столкновение игроков между собой	вызывается событие окончания игры
6	Работа кнопок меню (Host, Client, Exit)	корректно переключают состояние
7	Отображение победителя	отображается имя победителя после окончания сессии
8	Пауза через ESC	появляется и скрывается панель меню
9	Сохранение и загрузка настроек громкости	значения сохраняются и применяются при перезапуске
10	Перезапуск игры после завершения	текущая сессия завершена, клиент отключается корректно

По результатам тестирования были подтверждены:

- корректность сетевого взаимодействия между игроками (подключение, синхронизация, RPC);
- стабильная работа интерфейса, управление, пауза и переключение режимов;
- успешная работа логики еды и хвоста;
- корректное завершение игры и отображение результата.

Критических ошибок и сбоев в рамках проведённого тестирования не выявлено.

					09.02.07 7898.2025.У П.02.01	Лист 47
Изм	Лист	№ документа	Подпись	Дата		

Заключение

В ходе учебной практики был реализован проект многопользовательской 2D-игры **HEURO COIL**, вдохновлённой классической «змейкой» и адаптированной под современный сетевой формат. Основной целью являлась интеграция сетевого модуля и построение базовой клиентской логики в среде Unity.

На практике была успешно выполнена следующая работа:

- проведён анализ жанра и особенностей аркадных сетевых игр;
- спроектирована структура приложения с использованием методологии ICONIX;
- реализована основная механика: движение змейки, сбор еды, рост хвоста;
- внедрена сетевая библиотека **Mirror**, обеспечившая подключение игроков, синхронизацию объектов и игровую логику в режиме Host, Server и Client;
- настроено пользовательское меню с выбором режима, системой паузы и настройками звука.

Проект **не является полностью завершённым игровым продуктом**. В текущей версии отсутствует:

- реализация таблицы лидеров;
- система авторизации/регистрации;
- полноценное завершение интерфейса и визуальных эффектов;
- мобильная адаптация и оптимизация для маломощных устройств.

Тем не менее, в рамках практики был заложен **прочный архитектурный фундамент**, позволяющий при необходимости масштабировать и расширять функциональность проекта.

Главным результатом практики стало **освоение подходов к разработке многопользовательского приложения**, понимание клиент–серверной архитектуры, опыт работы с сетевыми событиями, синхронизацией объектов и взаимодействием между игроками.

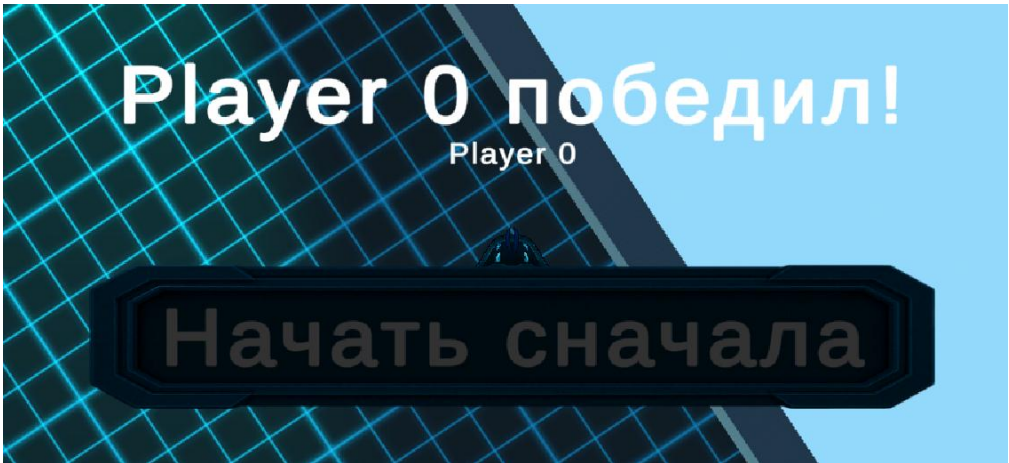
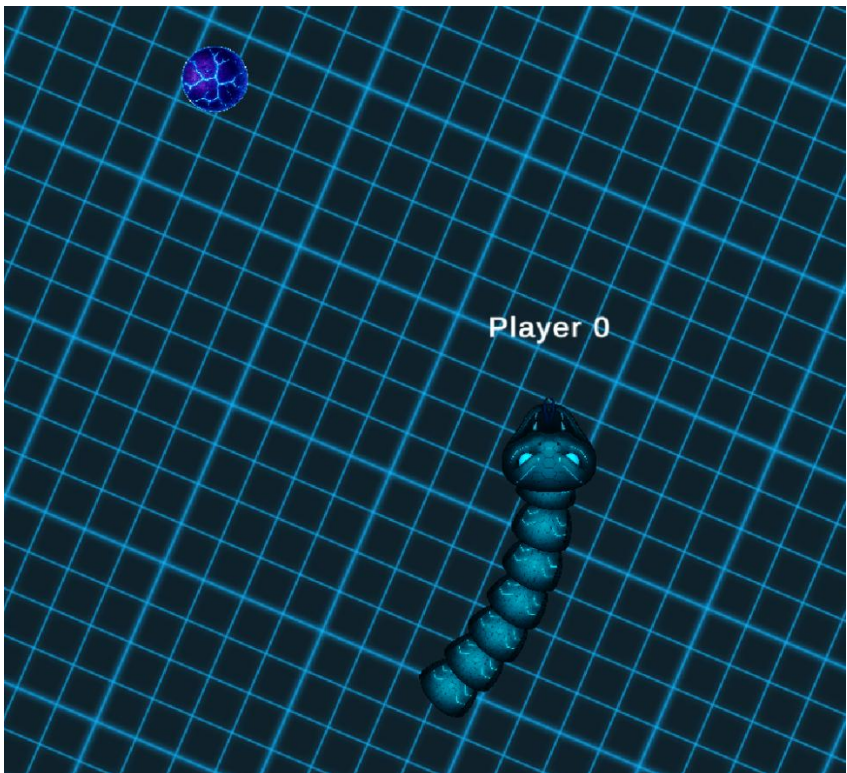
					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	48



					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	49



					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025. У П.02.01	50



					09.02.07	Лист
Изм	Лист	№ документа	Подпись	Дата	7898.2025.У П.02.01	51

Приложения

Приложение A.1 — GameOverDisplay.cs

Компонент отображения победителя. Отвечает за показ финального экрана и перезапуск игры после завершения сессии. Интегрирован с UI.

```
using TMPro;
using UnityEngine;
using Mirror;
using UnityEngine.SceneManagement;

public class GameOverDisplay : MonoBehaviour
{
    [SerializeField] private GameObject canvas;
    [SerializeField] private TMP_Text winnerNameText;

    void Start()
    {
        GameOverHandler.ClientOnGameOver += ClientHandleGameOver;

        if (canvas != null)
            canvas.SetActive(false);
        else
            Debug.LogWarning("GameOverDisplay: Canvas не назначен!");
    }

    void OnDestroy()
    {
        GameOverHandler.ClientOnGameOver -= ClientHandleGameOver;
    }

    void ClientHandleGameOver(string winner)
    {
        Debug.Log($"GameOverDisplay: Победитель - {winner}");

        if (canvas != null)
            canvas.SetActive(true);
        else
            Debug.LogError("GameOverDisplay: Canvas не найден!");

        if (winnerNameText != null)
            winnerNameText.text = $"{winner} победил!";
        else
            Debug.LogError("GameOverDisplay: WinnerNameText не назначен!");
    }

    public void RestartGame()
    {
        if (NetworkServer.active && NetworkClient.isConnected)
        {
            NetworkManager.singleton.StopHost();
        }
        else
        {
            NetworkManager.singleton.StopClient();
        }

        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
    }
}
```

Приложение A.2 — GameOverHandler.cs

Серверный обработчик завершения игры. Следит за подключёнными игроками и инициирует завершение сессии при достижении условия победы.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using Mirror;
using System;

public class GameOverHandler : NetworkBehaviour
{
    public static event Action<string> ClientOnGameOver;
    List<PlayerName> players = new List<PlayerName>();

    public override void OnStartServer()
    {
        PlayerSnake.ServerOnPlayerSpawned += ServerHandlePlayerSpawned;
        PlayerSnake.ServerOnPlayerDespawned += ServerHandlePlayerDespawned;
    }

    public override void OnStopServer()
    {
        PlayerSnake.ServerOnPlayerSpawned -= ServerHandlePlayerSpawned;
        PlayerSnake.ServerOnPlayerDespawned -= ServerHandlePlayerDespawned;
    }

    void ServerHandlePlayerSpawned(PlayerName player)
    {
        players.Add(player);
    }

    void ServerHandlePlayerDespawned(PlayerName player)
    {
        players.Remove(player);
        if (players.Count != 1) return;
        RpcGameOver(players[0].Name);
    }

    [ClientRpc]
    void RpcGameOver(string winner)
    {
        ClientOnGameOver?.Invoke(winner);
    }
}
```

Приложение A.3 — SnakeNetworkManager.cs

Расширенный менеджер сетевого взаимодействия.

Отвечает за спавн ключевых сетевых объектов — обработчика победы (GameOverHandler) и генератора еды (FoodSpawner).

```
using Mirror;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class SnakeNetworkManager : NetworkManager
```



```

{
    [SerializeField] GameObject foodSpawnerPrefab, gameOverHandlerPrefab;

    public override void OnStartServer()
    {
        var gameOverHandlerInstance = Instantiate(gameOverHandlerPrefab);
        NetworkServer.Spawn(gameOverHandlerInstance);
    }

    public override void OnServerAddPlayer(NetworkConnection conn)
    {
        base.OnServerAddPlayer(conn);
        var foodSpawnerInstance = Instantiate(foodSpawnerPrefab);
        NetworkServer.Spawn(foodSpawnerInstance);
    }
}

```

Приложение A.4 — SnakeMovement.cs

Компонент управления змейкой.

Реализует движение игрока и увеличение скорости при поедании еды. Подписан на сетевое событие ServerOnFoodEaten.

```

using System.Collections;
using System.Collections.Generic;
using Mirror;
using UnityEngine;

public class SnakeMovement : NetworkBehaviour
{
    [SerializeField] float rotationSpeed = 180f, speedChange = 0.5f;

    [SerializeField]
    [SyncVar]
    float speed = 3f;
    public float Speed
    {
        get { return speed; }
        private set { speed = value; }
    }

    public override void OnStartServer()
    {
        Food.ServerOnFoodEaten += ServerHandleFoodEaten;
    }

    public override void OnStopServer()
    {
        Food.ServerOnFoodEaten -= ServerHandleFoodEaten;
    }

    void ServerHandleFoodEaten(GameObject playerWhoAte)
    {
        if (gameObject == playerWhoAte)
            Speed += speedChange;
    }

    [ClientCallback]
    void Update()
    {
        transform.Translate(Vector3.forward * Speed * Time.deltaTime);
        transform.Rotate(Vector3.up * rotationSpeed * Time.deltaTime * Input.GetAxis("Horizontal"));
    }
}

```

Приложение A.5 — MyNetworkUI.cs

Скрипт пользовательского интерфейса подключения.

Управляет кнопками меню, переключающими сетевые режимы — Host, Client, Server и Local.

```
using Mirror;
using UnityEngine;
using UnityEngine.UI;

public class MyNetworkUI : MonoBehaviour
{
    [Header("Меню")]
    [SerializeField] private GameObject menuUI;
    [Header("Кнопки")]
    [SerializeField] private Button hostButton;
    [SerializeField] private Button clientButton;
    [SerializeField] private Button serverButton;
    [SerializeField] private Button localButton;

    private void Awake()
    {
        hostButton.onClick.AddListener(OnHostClicked);
        clientButton.onClick.AddListener(OnClientClicked);
        serverButton.onClick.AddListener(OnServerClicked);
        localButton.onClick.AddListener(OnLocalClicked);
    }

    private void OnHostClicked()
    {
        NetworkManager.singleton.StartHost();
        HideMenu();
    }

    private void OnClientClicked()
    {
        NetworkManager.singleton.StartClient();
        HideMenu();
    }

    private void OnServerClicked()
    {
        NetworkManager.singleton.StartServer();
        HideMenu();
    }

    private void OnLocalClicked()
    {
        HideMenu();
    }

    private void HideMenu()
    {
        menuUI.SetActive(false);
    }
}
```

Приложение A.6 — MenuManager.cs

Менеджер пользовательского интерфейса.

Реализует запуск игры, настройки громкости с сохранением, отображение меню паузы, затемнение экрана и выход из приложения.

```
using UnityEngine;
using UnityEngine.Audio;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using System.Collections;

public class MenuManager : MonoBehaviour
{
    [Header("Аудио и затемнение")]
    public AudioManager audioMixer;
    public float fadeDuration = 3f;
    public CanvasGroup fadeCanvasGroup;

    [Header("Меню паузы")]
    [SerializeField] private GameObject pauseMenuPanel;

    [Header("Слайдеры громкости")]
    [SerializeField] private Slider masterSlider;
    [SerializeField] private Slider musicSlider;
    [SerializeField] private Slider sfxSlider;

    private bool isPauseMenuVisible = false;

    private void Start()
    {
        float masterVolume = PlayerPrefs.GetFloat("MasterVolume", 1f);
        float musicVolume = PlayerPrefs.GetFloat("MusicVolume", 1f);
        float sfxVolume = PlayerPrefs.GetFloat("SFXVolume", 1f);

        SetVolume("MasterVolume", masterVolume);
        SetVolume("MusicVolume", musicVolume);
        SetVolume("SFXVolume", sfxVolume);

        if (masterSlider != null) masterSlider.value = masterVolume;
        if (musicSlider != null) musicSlider.value = musicVolume;
        if (sfxSlider != null) sfxSlider.value = sfxVolume;

        if (masterSlider != null) masterSlider.onValueChanged.AddListener((v) =>
SetVolumeAndSave("MasterVolume", v));
        if (musicSlider != null) musicSlider.onValueChanged.AddListener((v) =>
SetVolumeAndSave("MusicVolume", v));
        if (sfxSlider != null) sfxSlider.onValueChanged.AddListener((v) => SetVolume-
AndSave("SFXVolume", v));

        if (pauseMenuPanel != null)
            pauseMenuPanel.SetActive(true);

        StartCoroutine(FadeFromWhite());
    }

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            TogglePauseMenu();
        }
    }
}
```



```

private void TogglePauseMenu()
{
    if (pauseMenuPanel == null) return;

    isPauseMenuVisible = !isPauseMenuVisible;
    pauseMenuPanel.SetActive(isPauseMenuVisible);
}

private void SetVolume(string name, float value)
{
    float db = Mathf.Log10(Mathf.Clamp(value, 0.0001f, 1f)) * 20;
    audioMixer.SetFloat(name, db);
}

private void SetVolumeAndSave(string name, float value)
{
    SetVolume(name, value);
    PlayerPrefs.SetFloat(name, value);
}

private IEnumerator FadeFromWhite()
{
    if (fadeCanvasGroup == null)
    {
        Debug.LogWarning("fadeCanvasGroup не установлен!");
        yield break;
    }

    fadeCanvasGroup.alpha = 1f;
    fadeCanvasGroup.gameObject.SetActive(true);
    float timer = 0f;

    while (timer < fadeDuration)
    {
        timer += Time.deltaTime;
        fadeCanvasGroup.alpha = Mathf.Clamp01(1f - timer / fadeDuration);
        yield return null;
    }

    fadeCanvasGroup.alpha = 0f;
    fadeCanvasGroup.gameObject.SetActive(false);
}

private IEnumerator FadeAndLoadScene(int sceneIndex)
{
    if (fadeCanvasGroup == null)
    {
        Debug.LogWarning("fadeCanvasGroup не установлен!");
        yield break;
    }

    fadeCanvasGroup.gameObject.SetActive(true);
    fadeCanvasGroup.alpha = 0f;

    float timer = 0f;
    while (timer < fadeDuration)
    {
        timer += Time.deltaTime;
        fadeCanvasGroup.alpha = Mathf.Clamp01(timer / fadeDuration);
        yield return null;
    }

    fadeCanvasGroup.alpha = 1f;
    SceneManager.LoadScene(sceneIndex);
}

```

```

    public void Quit()
    {
        StartCoroutine(QuitCoroutine());
    }

    private IEnumerator QuitCoroutine()
    {
        yield return FadeAndLoadScene(0);
#if UNITY_EDITOR
        UnityEditor.EditorApplication.isPlaying = false;
#else
        Application.Quit();
#endif
    }
}

```

Приложение A.7 — TailSpawner.cs

Серверный компонент, отвечающий за добавление хвостовых сегментов змейки. Создает новые части хвоста при поедании еды, используя NetworkServer.Spawn().

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using Mirror;
using System;

public class TailSpawner : NetworkBehaviour
{
    [SerializeField] GameObject tailPrefab;
    public List<GameObject> Tails { get; } = new List<GameObject>();

    public override void OnStartServer()
    {
        Food.ServerOnFoodEaten += AddTail;
    }

    public override void OnStopServer()
    {
        Food.ServerOnFoodEaten -= AddTail;
    }

    void AddTail(GameObject playerWhoAte)
    {
        if (playerWhoAte != gameObject) return;
        var tailInstance = Instantiate(tailPrefab);
        NetworkServer.Spawn(tailInstance, connectionToClient);
    }
}

```

Приложение A.8 — TailNetwork.cs

Сетевой скрипт, определяющий, за кем следует хвост.

Каждый сегмент хвоста знает своего владельца и цель следования (голову или предыдущий хвост).

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using Mirror;

public class TailNetwork : NetworkBehaviour
{
    [SyncVar]

```

```

SnakeMovement owner;
public SnakeMovement Owner
{
    get { return owner; }
    private set { owner = value; }
}

[SyncVar]
GameObject target;
public GameObject Target
{
    get { return target; }
    private set { target = value; }
}

public override void OnStartServer()
{
    Owner = connectionToClient.identity.GetComponent<SnakeMovement>();
    var tails = Owner.GetComponent<TailSpawner>().Tails;
    Target = tails.Count == 0 ? Owner.gameObject : tails[tails.Count - 1];
    tails.Add(gameObject);
}
}

```

Приложение A.9 — TailMovement.cs

Класс движения хвоста.

Использует NavMeshAgent для плавного следования за целью (target) с учётом текущей скорости хозяина (owner).

```

using Mirror;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AI;

public class TailMovement : NetworkBehaviour
{
    [SerializeField] NavMeshAgent agent;
    [SerializeField] TailNetwork tail;

    void Start()
    {
        transform.position = tail.Target.transform.position;
    }

    void Update()
    {
        agent.speed = tail.Owner.Speed;
        agent.SetDestination(tail.Target.transform.position);
    }
}

```

Приложение A.10 — PlayerCameraController.cs

Скрипт активации камеры для локального игрока.

Включает камеру (cam.SetActive(true)) только для того экземпляра, у которого есть authority, исключая конфликты между клиентами.

```

using Mirror;
using System.Collections;
using System.Collections.Generic;

```

```

using UnityEngine;

public class PlayerCameraController : NetworkBehaviour
{
    [SerializeField] GameObject cam;

    public override void OnStartAuthority()
    {
        base.OnStartAuthority();
        cam.SetActive(true);
    }
}

```

Приложение A.11 — PlayerName.cs

Компонент отображения имени игрока.

Генерирует имя на сервере по ID подключения и синхронизирует его через SyncVar с hook, чтобы обновить текстовое поле (TMP_Text) на клиенте.

```

using Mirror;
using System;
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class PlayerName : NetworkBehaviour
{
    [SerializeField] TMP_Text playerNameText;
    [SyncVar(hook = nameof(HandlePlayerNameUpdated))]
    string playerName;

    public string Name { get { return playerName; } }

    void HandlePlayerNameUpdated(string oldText, string newText)
    {
        playerNameText.text = newText;
    }

    public override void OnStartServer()
    {
        playerName = $"Player {connectionToClient.connectionId}";
    }
}

```

Приложение A.12 — PlayerSnake.cs

Главный игровой скрипт игрока.

Содержит логику появления, столкновений и удаления игрока. Обрабатывает взаимодействия со стенами, хвостами и другими игроками. Вызывает события ServerOnPlayerSpawned и ServerOnPlayerDespawned для управления победой.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using Mirror;
using System;

public class PlayerSnake : NetworkBehaviour
{
    [SerializeField] TailSpawner tailSpawner;
    [SerializeField] PlayerName playerName;
    public static event Action<PlayerName> ServerOnPlayerSpawned;
    public static event Action<PlayerName> ServerOnPlayerDespawned;
}

```

```

public override void OnStartServer()
{
    ServerOnPlayerSpawned?.Invoke(playerName);
}

[ServerCallback]
void OnTriggerEnter(Collider other)
{
    if (other.TryGetComponent(out NetworkIdentity networkIdentity)
        && networkIdentity.connectionToClient == connectionToClient)
        return;
    switch (other.tag)
    {
        case "Border":
        case "Player":
        case "Tail":
            DestroySelf();
            break;
    }
}

void DestroySelf()
{
    ServerOnPlayerDespawned?.Invoke(playerName);
    foreach (var tail in tailSpawner.Tails)
        NetworkServer.Destroy(tail);
    NetworkServer.Destroy(gameObject);
}
}

```

Приложение A.13 — Food.cs

Серверный скрипт обработки столкновения с едой.

Вызывает событие ServerOnFoodEaten при попадании в коллайдер игрока, уничтожает объект еды и запускает эффект (при наличии particlePrefab).

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using Mirror;
using System;

public class Food : NetworkBehaviour
{
    [SerializeField] GameObject particlePrefab;

    public static event Action<GameObject> ServerOnFoodEaten;

    void OnTriggerEnter(Collider other)
    {
        if (!other.CompareTag("Player")) return;
        ServerParticles();

        NetworkServer.Destroy(gameObject);
        ServerOnFoodEaten?.Invoke(other.gameObject);
    }

    [ServerCallback]
    void ServerParticles()
    {
        GameObject boom = Instantiate

```

```

        (particlePrefab, transform.position, particlePrefab.transform.ro-
tation);
        NetworkServer.Spawn(boom);
        StartCoroutine(DelayedDestroy(boom, 3f));
    }

    IEnumerator DelayedDestroy(GameObject obj, float delay)
    {
        yield return new WaitForSeconds(delay);
        NetworkServer.Destroy(obj);
    }
}

```

Приложение A.14 — FoodSpawner.cs

Компонент, размещающий еду на сцене.

При запуске сервера генерирует первую еду, а затем реагирует на событие ServerOnFoodEaten, создавая новую в случайной позиции. Обеспечивает непрерывное появление еды.

```

using Mirror;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class FoodSpawner : NetworkBehaviour
{
    [SerializeField] GameObject foodPrefab;
    [SerializeField] float xSize = 8f, zSize = 8f;

    public override void OnStartServer()
    {
        SpawnFood(gameObject);
        Food.ServerOnFoodEaten += SpawnFood;
    }

    public override void OnStopServer()
    {
        Food.ServerOnFoodEaten -= SpawnFood;
    }

    [Server]
    void SpawnFood(GameObject playerWhoAte)
    {
        var pos = new Vector3(
            Random.Range(-xSize, xSize),
            foodPrefab.transform.position.y,
            Random.Range(-zSize, zSize));
        var foodInstance = Instantiate(foodPrefab, pos, foodPrefab.transform.rota-
tion);
        NetworkServer.Spawn(foodInstance);
    }
}

```